Sesión 2

Copy

Curso Azure AI

Contenidos de Hoy

1. Azure OpenAI
2. MAF
3. Descanso
4. LangGraph, LangChain, MAF & Foundry
5. Proyectos
6. Recap y Dudas
7. Cuestionario

`git pull --rebase` en curso-azureai

AZURE OPENAI COMO NÚCLEO DE LOS AGENTES

Servicios de Azure OpenAI como base de inferencia dentro de la arquitectura agéntica

Azure OpenAI aporta el **motor de inferencia**: acceso a modelos de lenguaje, visión e embeddings en entorno Azure, con APIs compatibles y con la capa de seguridad y operación empresarial de Azure. Enc[illegible] dentro de Azure, incluyendo familias como [illegible]s, y destaca que co-desarrolla las APIs con C[illegible]sición fluida entre ecosistemas.

This toolbar appears only when signed in to GitBook. Dismiss
Curso Azure AI
0 minutes ago

Qué lugar ocupa Azure OpenAI dentro del stack

Azure OpenAI vive principalmente en **la capa de modelos e inferencia**.

- **Azure OpenAI** resuelve inferencia y acceso a modelos.
- **Foundry** añade plataforma para construir, evaluar y operar.
- **MAF / LangGraph / LangChain** estructuran comportamiento, estado y flujo.
- **Azure AI Search, datos y tools** aportan grounding y acción.

Qué decisiones arquitectónicas dependen de Azure OpenAI

Decisión 1. Qué modelo usar según la tarea

No todas las tareas necesitan el mismo modelo.

Azure OpenAI en Foundry Models ofrece distintos modelos con capacidades y puntos de precio diferentes, y que la disponibilidad varía por región. Eso obliga a elegir según:

- complejidad,
- latencia,
- coste,
- modalidad,
- y riesgo del caso.

Decisión 2. Cómo organizar deployments

Conviene pensar en deployments por responsabilidad arquitectónica, por ejemplo:

- uno para conversación,
- otro para clasificación,
- otro para embeddings,
- otro para fallback o tareas críticas.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Decisión 3. Cómo actualizar versiones sin romper comportamiento

Microsoft mantiene una política de **model retirements** y políticas de upgrade. Por tanto, cambiar versión o familia de modelo no debe tratarse como un cambio trivial: puede alterar comportamiento, calidad, coste y compatibilidad.

Decisión 4. Qué API usar

Microsoft ya documenta que la **Responses API** es la vía unificada para funcionalidades más recientes, mientras que **Chat Completions** sigue existiendo como camino conocido para modelos conversacionales.

Sin embargo, **Responses API no está disponible en todas las regiones**

Conceptos clave

> **Responses API**
>
> API más reciente y unificada de Azure OpenAI para funcionalidades modernas; Microsoft la posiciona como evolución de experiencias antes separadas.

> **Chat completions**
>
> API conversacional conocida y todavía relevante para muchos escenarios de chat.

Uso de modelos en agentes según tarea, complejidad, latencia y coste esperado

La regla correcta no es “usar siempre el modelo más potente”, sino **usar el modelo adecuado para el trabajo adecuado**. La documentación oficial de Microsoft diferencia claramente entre familias orientadas a **reasoning profundo** y familias orientadas a **respuesta rápida y alto throughput**.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Por ejemplo, **GPT-5** está orientado a razonamiento estructurado, lógica multistep y planificación, mientras que **GPT-4.1** está optimizado para chat en tiempo real, soporte, resumido ligero y cargas de alto volumen con menor latencia

Primer criterio: la tarea que realiza el agente

La tarea es el criterio más importante.

1. Chat en tiempo real, soporte y Q&A rápido

Para agentes de atención, soporte o consulta rápida, Microsoft recomienda GPT-4.1 cuando importan **baja latencia**, **alto throughput** y respuestas concisas. La guía oficial lo pone como mejor ajuste para customer service assistants y workloads de alto volumen.

2. Planificación, análisis y tool calling complejo

Cuando el agente tiene que:

- planificar,
- decidir entre varias tools,
- hacer razonamiento multi-hop,
- o coordinar pasos con más profundidad,

Microsoft posiciona GPT-5 como mejor opción, precisamente por su mayor capacidad de reasoning y planificación en workflows agentic.

3. Clasificación, routing y tareas ligeras

Para clasificación, triage, etiquetado o decisiones sencillas, lo normal es que no compense usar el modelo más caro o más lento.

4. Embeddings y recuperación semántica

Cuando la tarea no es “responder”, sino **representar semánticamente** información para búsqueda, similitud o RAG, el tipo de modelo cambia por completo: aquí lo correcto son **embeddings**, no un modelo conversacional.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Segundo criterio: latencia esperada

GPT-5 ofrece más capacidad de reasoning, pero con **mayor TTFT** y mayor latencia percibida; GPT-4.1 ofrece una experiencia más “ágil” y está pensado para workloads sensibles a la latencia.

Microsoft también publica métricas como **TTFT**, **GTPS**, **TTPS** y percentiles de latencia en sus benchmarks y documentación de rendimiento.

TTFT incluye:

- tiempo de clasificación
- búsqueda RAG
- preparación del prompt
- cola del deployment
- primer token del modelo

GTPS mide:

- velocidad de generación una vez empieza la respuesta

TTPS mide:

- eficiencia total del request incluyendo prompt + respuesta

Provisioned throughput es una opción cuando tienes requisitos bien definidos de throughput y latencia, y dice explícitamente que esta modalidad está pensada para aplicaciones de producción con patrones predecibles y necesidades de rendimiento más estables.

- **Interacción usuario en vivo**: prioriza latencia y consistencia.
- **Procesamiento batch o background**: puedes tolerar más latencia si ganas calidad.
- **Tiempo real crítico**: considera provisioned throughput si el tráfico es estable y el caso justifica el coste.

Se ejecuta reservando **PTU's (Provisioned Throughput Units)**

Por ejemplo para GPT-4o, 1 PTU equivaldría aprox. a 2.500 tokens de entrada/minuto y 2.000 tokens de salida/minuto, aunque depende de tools, etc. y su coste sería

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Modalidad	Mínimo	Precio unitario	Coste mínimo aproximado
Global Provisioned	15 PTUs	$1/PTU/hora	**$15/hora** o **~$10.950/mes** si está 24/7 sin reserva
Global con reserva mensual	15 PTUs	$260/PTU/mes	**$3.900/mes**
EU/US Data Zone con reserva mensual	15 PTUs	$286/PTU/mes	**$4.290/mes**
Regional con reserva mensual	50 PTUs	$286/PTU/mes	**$14.300/mes**

Tercer criterio: coste esperado

El coste real depende del ratio input/output, del patrón de uso y del tipo de deployment. En los leaderboards, el coste estimado se calcula con un ratio 3:1 entre input y output.

Costes ↗

Cuarto criterio: throughput y escala

GPT-4.1 está optimizado para **high-throughput enterprise applications**, mientras que GPT-5 tiene throughput más moderado por su mayor profundidad de reasoning.

Conceptos clave

> **Modelo de reasoning**
>
> Modelo que dedica más tiempo a procesar y entender la petición, con ventaja en tareas complejas de análisis, coding y planificación.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

> **Provisioned throughput**
>
> Modalidad de despliegue que reserva capacidad para obtener rendimiento más estable y predecible en workloads de producción con patrones conocidos.

[Modelos Actuales ↗]

[Leaderboard ↗]

Ejercicio. Selección de Modelos

Buscar modelos recientes para:

- soporte chat en vivo,
- clasificación de incidencias,
- agente planificador con varias tools,
- embeddings para RAG,
- resumen ejecutivo de documentos largos.

Gestión de endpoints y deployments para organizar despliegues de forma robusta

En Azure OpenAI el consumo real se hace contra **deployments**, no contra el nombre abstracto del modelo.

Además, los **endpoints** de Azure OpenAI trabajan a nivel de deployment y cada deployment tiene su propia URL derivada del endpoint base del recurso.

Dicho de otra manera:

- el **modelo** define capacidad,
- el **deployment** define
- y el **endpoint** define

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

El **endpoint** de Azure OpenAI suele ser de la forma `https://<resource-name>.openai.azure.com` y cada **deployment** se consume con una ruta basada en `/deployments/<deployment-name>`.

Un deployment no es solo un alias: encapsula **versión del modelo, tipo de capacidad y otras configuraciones** operativas

Buenas prácticas

1. Separar deployments por función

No conviene usar un único deployment para todo. Lo normal es separar, como mínimo:

- chat general,
- reasoning más costoso,
- embeddings,
- clasificación o tareas ligeras,
- y, si hace falta, uno de fallback.

2. Separar deployments por entorno

Una gestión robusta exige aislar **dev**, **test** y **prod**. Azure OpenAI trata los deployments como recursos Azure sujetos a políticas y cuota, así que mezclar entornos en el mismo naming o en el mismo flujo operativo complica auditoría, pruebas y rollback.

3. Usar nombres lógicos y estables

Un buen nombre de deployment describe **función** y **entorno**, no solo el modelo. Ejemplos razonables:

- `chat-prod`
- `reasoning-prod`
- `embed-prod`
- `chat-dev`

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

4. Elegir bien el tipo de deployment

Microsoft distingue dos grandes categorías de deployment:

- **Standard**, con pago por token.
- **Provisioned**, con capacidad reservada

Además, dentro de esas categorías puedes elegir variantes según residencia de datos y patrón operativo:

- **Global Standard / Global Provisioned**, si no tienes restricciones geográficas fuertes y quieres más disponibilidad inicial o más capacidad.
- **Data Zone Standard / Data Zone Provisioned**, si necesitas que prompts y responses se procesen dentro de la zona EU o US.
- **Regional Standard / Regional Provisioned**, si tu requisito es una región concreta.

5. Cuotas y rate limits

Las cuotas de Azure OpenAI **no** se aplican a nivel de tenant, sino a nivel de **suscripción**, **región** y **modelo/tipo de deployment**. La cuota se expresa en **TPM** y debes gestionarla entre deployments dentro de la suscripción.

- **Cuota máxima disponible**: la fija **Microsoft/Azure** para tu **suscripción + región + modelo + tipo de deployment**. Por ejemplo: `Subscription A + West Europe + GPT-4o + Global Standard`. Esa cuota se expresa normalmente en **TPM** para Standard o en **PTU** para Provisioned.
- **Cuota asignada al deployment**: la fija quien crea o administra el deployment dentro del límite aprobado. Es decir, si tu suscripción tiene 240K TPM para GPT-4o en una región, puedes repartirlo entre uno o varios deployments mientras la suma no supere el total.

Para ver deployments existentes de un recurso Azure OpenAI (con Rol Usages Reader):

```
 az cognitiveservices account deployment list --resource-group Curso-AI --name
curso-ia-openai-project --output json
```

Para ver un deployment concreto:

```
az cognitiveservices                                                    Curso-AI --name
curso-ia-openai-proje
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Buen patrón de diseño

- `chat-prod` para conversación normal.
- `reasoning-prod` para tareas complejas o tool orchestration delicada.
- `embed-prod` para RAG.
- `chat-canary` para validar una nueva versión de modelo.
- `chat-dr` o duplicado regional para continuidad de negocio.

Conceptos clave

> **Endpoint**
>
> Punto de acceso HTTP del recurso Azure OpenAI o Foundry Models. En Azure OpenAI, se consume inferencia a partir del endpoint base del recurso más la ruta del deployment.

> **Deployment**
>
> Recurso operativo que publica un modelo con una configuración concreta de versión, capacidad y filtros. La aplicación llama al deployment, no al modelo abstracto.

Ejemplo: fallback entre endpoint primario y secundario

.env (en raiz)

```
AOAI_ENDPOINT_PRIMARY="https://curso-ia-openai.openai.azure.com/"
AOAI_DEPLOYMENT_CHAT_PRIMARY="gpt-4okk"
AOAI_API_KEY_PRIMARY=

AOAI_ENDPOINT_SECONDARY="https://curso-ia-openai-responses.openai.azure.com/"
AOAI_DEPLOYMENT_CHAT_SECONDARY="gpt-4o"
AOAI_API_KEY_SECONDARY=
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

build_client.py

```
import os
from typing import Optional

from openai import AzureOpenAI, OpenAI, OpenAIError
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
from azure.core.exceptions import AzureError, ClientAuthenticationError


DEFAULT_API_VERSION = os.getenv("AZURE_OPENAI_API_VERSION", "2024-10-21")
```

az201.py

```
import os
from typing import List

from dotenv import load_dotenv

from build_client import build_client_chat


# Carga variables desde .env
load_dotenv("../.env")
```

Ejercicio: Model Escalation

Adapta el script az201.py para que:

- Lea un fichero ticket.txt, que contiene una incidencia.
- Lo intenta clasificar en varias categorías ["Bug", "Config", "Infraestructura", "Otras"] con un modelo sencillo (Deployment Primario/gpt-4o-mini)
- Si el resultado es "Otras" o inválido, lo escala a un modelo mas complejo (Deployment Secundario/gpt-5.1-chat)

Control de versiones de modelos y estrategia de actualización sin romper comportamiento

En Azure OpenAI, el probl **evito que mañana cambie el comportamie**

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Cuando aparece una nueva versión de un modelo, puedes probarla inmediatamente en nuevos deployments; también notifica con al menos dos semanas de antelación antes de que una nueva versión pase a ser la versión por defecto, y mantiene la versión mayor previa hasta su fecha de retirada.

Además, cuando un modelo llega a retirement, el efecto depende de la política configurada en el deployment.

Qué significa “versión”

1. Versión del modelo
Es la versión concreta de pesos o snapshot que soporta ese deployment. Azure publica nuevas versiones, permite probarlas en deployments nuevos y mantiene políticas de upgrade por deployment.

2. Versión de API
Es el contrato técnico con el que tu código habla con el servicio.

Una nueva versión de modelo no siempre obliga a cambiar la versión de API, porque a veces solo cambian los pesos y no el contrato; pero también aclara que, en otras transiciones, sí puede cambiar la interfaz y conviene revisar compatibilidad.

Qué políticas de actualización existen

Auto-update to default (`OnceNewDefaultVersionAvailable`)
Cuando una nueva versión pasa a ser la versión por defecto, el deployment se actualiza automáticamente en un plazo de hasta dos semanas.

En preview, la actualización se aplica cuando aparece una nueva preview, también con esa ventana. Microsoft recomienda esta opción especialmente en fases tempranas de prueba.

Upgrade when current version expires (`OnceCurrentVersionExpired`)
El deployment se mantiene en su versión actual y solo se actualiza cuando esa versión llega a retirement; si la propiedad no aparece, Microsoft indica que el comportamiento efectivo es éste.

No auto upgrade (`NoAutoUpgrade`)
El deployment nunca se actualiza automáticamente. Cuando llega la fecha de retirada, deja de funcionar y devuelve errores hasta que lo migres a una versión soportada.

This toolbar appears only when signed in to GitBook.

Estas políticas automátic **tándar**, no para todos los tipos de despliegue.

Curso Azure AI
0 minutes ago

Qué puede romper una actualización aunque “todo siga funcionando”

Aunque el deployment siga respondiendo, una actualización puede cambiar varias cosas relevantes:

Puede cambiar la **calidad** de las respuestas, la **consistencia** del formato, la forma en que sigue instrucciones, la precisión del **tool calling**, la **latencia**, el **coste** o incluso el ajuste fino de seguridad y groundedness que tu pipeline esperaba.

> El peligro no es solo que el deployment caiga.
> El peligro es que siga funcionando, pero **de otra manera**.

Cómo afecta el lifecycle oficial del modelo

Existen varias etapas de ciclo de vida para modelos en Foundry Models: **preview, GA, legacy, deprecated y retired**.

En general, los modelos GA son adecuados para producción; los deprecated ya no aceptan nuevos deployments, pero los existentes siguen funcionando hasta retirement; y los retired dejan de servir tráfico y devuelven errores.

Las versiones GA están disponibles un mínimo de 12 meses, que los clientes existentes pueden seguir usando una versión antigua durante 6 meses adicionales, y se da al menos 60 días de aviso antes del retirement de modelos GA.

Estrategia recomendada para actualizar sin romper comportamiento

1. Separar deployments por propósito

2. Mantener al menos dos líneas de deployment

3. Validar con corpus real

4. Decidir la política de upgrade según criticidad

En desarrollo o exploración puede tener sentido `Auto-update to default`. En producción, muchas veces encaja mej[illegible]orma controlada, o dejar `OnceCurrentVersionExpir[illegible]`

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

5. Tener rollback o ruta alternativa

Qué hacer con deployments provisionados

En despliegues **provisioned**, la historia cambia un poco.

La gestión de actualizaciones y migraciones en provisioned deployments se trata de forma específica, y soporta tanto **migraciones in-place** a una nueva versión dentro de la misma familia como **multi-deployment migrations**, donde creas un deployment nuevo y mueves tráfico gradualmente.

Parámetros de inferencia y cómo afectan a calidad, consistencia y coste

Los parámetros de inferencia **regulan su comportamiento** dentro de una tarea concreta. No conviene pensar en ellos como “ajustes mágicos”, sino como controles de:

- **muestreo**,
- **longitud de salida**,
- **formato**,
- **grado de razonamiento**,
- y **reproducibilidad**.

Qué familias de parámetros conviene entender primero

1. **Variabilidad y consistencia**
   `temperature`, `top_p`, `seed`
2. **Longitud y presupuesto de salida**
   `max_tokens`, `max_completion_tokens`, `max_output_tokens`, `stop`, `n`
3. **Repetición y exploración temática**
   `frequency_penalty`, `presence_penalty`
4. **Formato y estructura de la respuesta**
   `response_format` con `text`, `json_object` o `json_schema`
5. **Parámetros específicos de reasoning**
   `reasoning_effort`, [illegible] `soning.summary`

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Parámetros que más afectan a consistencia

1. `temperature`

`temperature` es el parámetro que incrementa la **aleatoriedad** de la salida cuando sube su valor. En términos prácticos:

- **baja temperatura** → respuestas más predecibles y estables,
- **alta temperatura** → respuestas más diversas y creativas.

2. `top_p`

Es una alternativa a `temperature` para **nucleus sampling**: con `top_p=1.0` el muestreo considera todo el conjunto de tokens posibles; al bajarlo, restringes el espacio de salida a la parte más probable.

En práctica, suele ser más limpio **ajustar uno primero** y no mover ambos a la vez sin medir resultados.

3. `seed`

Azure OpenAI soporta `seed` como parámetro para hacer un **best effort** de **.py**. Microsoft deja claro que:

- está en **beta/preview** según la superficie de API,
- la **determinación no está garantizada**,
- y conviene observar también `system_fingerprint` para detectar cambios del backend. Además, Microsoft indica que respuestas con `max_tokens` altos tienden a ser **menos deterministas** incluso usando `seed`.

Parámetros que más afectan a coste y longitud

4. `max_tokens`

This toolbar appears only when signed in to GitBook.

`max_tokens` fija el máxim **ontrolar coste**. También recuerda que prompt + ntexto del modelo.

Curso Azure AI
0 minutes ago

5. `max_completion_tokens` y `max_output_tokens`

Aquí hay un matiz muy importante y bastante actual:

- para **reasoning models** con **Chat Completions API**, Microsoft indica usar `max_completion_tokens`,
- para **Responses API**, indica usar `max_output_tokens`.

`max_completion_tokens` es un límite superior para tokens de la completion, incluyendo **visible output tokens y reasoning tokens**.

6. `stop`

`stop` permite definir hasta **cuatro secuencias** donde la generación debe detenerse, pero también aclara que **no está soportado** en los modelos de reasoning más recientes `o3` y `o4-mini`.

7. `n`

`n`, al generar varias completions por prompt, puede **consumir cuota rápidamente**, y recomienda usarlo con cuidado junto con `max_tokens` y `stop`.

Parámetros que afectan a repetición, estructura y control

8. `frequency_penalty` y `presence_penalty`

La referencia define:

- `frequency_penalty`: penaliza tokens según su frecuencia previa, reduciendo repetición literal.
- `presence_penalty`: penaliza tokens por haber aparecido ya, empujando al modelo a hablar de temas nuevos.

9. `response_format`

This toolbar appears only when signed in to GitBook.

Tres rutas principales:

Curso Azure AI
0 minutes ago

- `text`
- `json_object`
- `json_schema`

Además:

- `json_object` es el modo JSON **más antiguo**,
- `json_schema` es el enfoque **preferido** cuando el modelo lo soporta, porque permite **Structured Outputs** que ajustan la salida a tu esquema.

Con `json_object`, debes **instruir explícitamente** al modelo para devolver JSON; si no, puede producir una salida problemática o aparentemente bloqueada.

Parámetros específicos de modelos de reasoning

10. `reasoning_effort`

`reasoning_effort` para modelos de razonamiento con valores como `none`, `minimal`, `low`, `medium`, `high` y `xhigh`, y dice explícitamente que **reducir el effort puede dar respuestas más rápidas y usar menos tokens de razonamiento**.

11. `verbosity`

La referencia documenta `verbosity` con valores `low`, `medium` y `high`, y la describe como un control del nivel de **concisión o detalle** de la respuesta. La guía de reasoning muestra además `text={"verbosity":"low"}` como parámetro nuevo en modelos GPT-5

12. `reasoning.summary`

En Responses API puedes pedir un **resumen del razonamiento** mediante el parámetro correspondiente, pero también deja claro que intentar extraer raw chain-of-thought por otros medios no está soportado.

Cómo afectan de verdad a calidad, consistencia y coste

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Calidad

Los parámetros que más afectan a calidad útil en agentes suelen ser:

- `response_format`
- `max_tokens` / `max_output_tokens`
- `reasoning_effort` en tareas complejas
- y, solo después, `temperature` / `top_p`.

Consistencia

Los parámetros más relevantes para consistencia son:

- `temperature`
- `top_p`
- `seed`
- `response_format`
- y límites de salida bien ajustados.

Coste

Los parámetros con impacto más directo sobre coste son:

- `max_tokens` / `max_completion_tokens` / `max_output_tokens`
- `n`
- `reasoning_effort`
- y, en menor medida, `verbosity`.

> **Temperatura y top_p** regulan variabilidad.
> **Max tokens y n** regulan presupuesto.
> **Response format** regula estructura.
> **Reasoning effort y verbosity** regulan profundidad y extensión.
> **Seed** ayuda a comparar, no a garantizar igualdad absoluta.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Diferencias entre Modelos e Interfaces

Caso	GPT-4o / GPT-4 clásico	G re
Control de creatividad	`temperature`, `top_p`	N `t` `re` e re u `t` `_` in e `v` y
Límite de salida en Chat	`max_tokens` todavía suele funcionar, aunque mejor migrar	`m` e n
Límite de salida en Responses	`max_output_tokens`	`m` t
Razonamiento controlado	No aplica como parámetro principal	`t` `_` C `t` = { . R
Concisión	Prompt + `max_tokens`	`t` { y " m e R

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

JSON	`response_format`	M `t` a R o e si s

Ejemplo: salida estructurada en completions

```
# JSON sin Schema
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "Analiza la incidencia y devuelve un objeto
JSON válido."},
        {"role": "user", "content": "El cliente no puede facturar desde ayer y hay
urgencia alta."}
    ],
    response_format={"type": "json_object"},
    temperature=0.2,
```

Ejercicio.

Sobre el clasificador de incidencias:

- Extraer también "Prioridad"
- Probar en texto plano, con json_object y con json_schema y ver efectividad
- Probar con distintos modelos
- Tunear parámetros como temperatura, max_tokens u otros

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: razonamiento en Responses

az202.py

```
import os
from pathlib import Path

from dotenv import load_dotenv

from build_client import build_client_resp


load_dotenv("../.env")
```

Ejercicio.

Sobre el clasificador de incidencias:

- Aplicar razonamiento para extraer un plan de acción
- Tunear distintos parámetros

Costes y latencia como variables de diseño y no solo de operación

El **throughput** es la capacidad global del deployment medida en peticiones y tokens por minuto; la latencia es el tiempo que tarda una llamada concreta en empezar a responder o en completarse.

Para chat/completions, la latencia depende sobre todo del **modelo**, del **número de tokens del prompt**, del **número de tokens generados** y de la **carga total** del sistema; y añade un matiz muy importante: en general, **cada token generado añade más tiempo que cada token del prompt**.

Qué variables influyen más en latencia

1. el **modelo**,
2. el **número de tokens**
3. el **número de tokens g**

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

4. y la **carga global** del deployment y del sistema.

Los factores 1 y 3 suelen ser los más determinantes, y reducir tokens generados es una de las maneras más directas de reducir latencia.

Qué variables influyen más en coste

Para estimar throughput de un workload hay que observar al menos dos métricas: **Processed Prompt Tokens** y **Generated Completion Tokens**. Cuando las sumas a escala de peticiones por minuto, obtienes una aproximación útil del consumo real del deployment.

En diseño, las palancas más claras son estas:

1. Tokens de entrada

Cuanto más largo es el prompt, más tokens procesas. Aunque su impacto en latencia suele ser menor que el de la generación, sí impactan directamente en coste total.

2. Tokens de salida

Son la palanca más peligrosa porque suben a la vez:

- coste,
- latencia,
- y riesgo de verbosidad innecesaria.

Reducir el número de tokens generados mejora la latencia y, por extensión, reduce consumo total.

3. Elección de modelo

GPT-4.1 está más orientado a coste-eficiencia y throughput alto en tareas generales, mientras que GPT-5 asume más coste y latencia para ganar profundidad de razonamiento.

Throughput, cuota y por qué no todo problema de latencia se arregla "subiendo límites"

En deployments estándar, la cuota influye en la admisión de llamadas, pero **no impone directamente el throughp** da, no siempre alcanzarás tanto throughp

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

En los usage tiers de Azure OpenAI, mientras te mantienes dentro del tier asignado la latencia es más estable; pero si lo excedes, la latencia puede aumentar mucho e incluso llegar a ser **más del doble** en algunos casos, además de aparecer errores `429`.

Diseñar para latencia: decisiones concretas

1. Separar workloads

Mezclar workloads distintos en el mismo endpoint puede empeorar latencia por dos motivos:

- las peticiones se batchéan juntas y las cortas pueden esperar a las largas,
- y además baja el **cache hit rate** porque compiten por el mismo espacio.
  Por eso recomienda separar deployments por workload cuando sea posible.

2. Medir bien streaming y no streaming

- **End-to-end Request Time** para no streaming,
- **Time to Response** para streaming,
- y **Average Token Generation Rate** para medir velocidad de generación.

Diseñar para coste: decisiones concretas

1. Prompt caching

Es una forma de reducir **latencia y coste** cuando varias peticiones comparten un prefijo largo idéntico. Para que aplique, la solicitud debe tener al menos **1.024 tokens** y los primeros **1.024 tokens** deben ser idénticos. En modelos soportados, los tokens cacheados tienen descuento en standard y pueden tener hasta **100% de descuento en input tokens** en provisioned.

2. Predicted outputs

Cuando gran parte de la respuesta ya es conocida, por ejemplo en refactorización o edición de texto, se puede usar parámetro `predicted`. Está en preview, para ciertos modelos y no siempre reduce coste: los `accepted_prediction_tokens` ayudan, pero los `rejected_prediction_tokens` pueden encarecer la llamada, así que hay que evaluar el balance caso por caso.

3. Provisioned throughput

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Provisioned throughput tiene sentido cuando el patrón de tráfico es conocido y estable, y necesitas **latencia máxima estable** y **capacidad asignada**. Se recomienda explícitamente para aplicaciones de producción con throughput y latencia previsibles, especialmente si son sensibles a la latencia.

4. Spillover

Cuando hay picos en **provisioned**, el tráfico sobrante puede redirigirse a un deployment estándar asociado, reduciendo interrupciones en bursts. Esto es muy útil si quieres mezclar estabilidad y elasticidad sin sobredimensionar siempre el provisionado.

Conceptos clave

Latencia

Tiempo que tarda el modelo en responder a una llamada. Para streaming conviene distinguir **time to response** y **average token generation rate**.

Throughput

Capacidad global del deployment, medida en peticiones y tokens por minuto. Microsoft recomienda observar input TPM y output TPM por separado para entender mejor el workload.

Usage tier

En deployments estándar, define el throughput máximo esperable con latencia predecible para la mayoría de clientes de tráfico bajo o medio. Excederlo puede aumentar mucho la variabilidad de latencia.

Provisioned throughput

Capacidad reservada con rendimiento más estable y predecible para workloads uniformes, especialmente útil en producción con necesidades conocidas de throughput y latencia

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Prompt caching

Mecanismo para reducir latencia y coste en prompts largos con prefijos idénticos.

Spillover

Mecanismo que desvía tráfico excedente desde un deployment provisionado a uno estándar para absorber picos.

Ejemplo: diseñar un prompt cache-friendly

```
SYSTEM_PREFIX = """
Eres un asistente de soporte interno.
Sigue siempre las políticas corporativas.
Usa el formato JSON cuando el flujo lo requiera.
No inventes datos.
""".strip()

def build_prompt(user_question: str, dynamic_context: str) -> str:
    # Contenido repetido primero, contenido variable después
    return f"
{SYSTEM_PREFIX}\n\nContexto:\n{dynamic_context}\n\nPregunta:\n{user_question}"
```

Seguridad y aislamiento para trabajar con datos corporativos en contexto regulado

En contexto regulado, la pregunta no es solo “¿el modelo responde bien?”, sino:

> **¿dónde se procesa el dato, quién puede acceder, por qué red circula, con qué clave se cifra y qué controles reducen fuga o manipulación?**

Los **prompts y respuestas no se usan para entrenar ni mejorar los modelos base**, que los modelos son **stateless**, y que los datos en reposo se almacenan dentro de la geografía del recurso con **cifrado AES-256 por defecto**, con opción de **customer-managed keys** en escenarios soportados.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Qué significa realmente “contexto regulado”

No hace falta que una organización pertenezca a un sector extremo para que este tema importe. Basta con que el agente trabaje con:

- datos de clientes,
- datos de empleados,
- documentación contractual,
- contenido financiero,
- información sanitaria,
- o propiedad intelectual relevante.

En esos casos, la arquitectura debe demostrar control sobre:

- **residencia del dato**
- **segregación de acceso**
- **cifrado**
- **rutas de red**
- y **medidas de reducción de riesgo**.

Primer principio: elegir bien el tipo de deployment según residencia y procesamiento

Uno de los puntos más importantes, y a menudo peor entendidos, es que **“dónde está el recurso”** y **“dónde puede procesarse la inferencia”** no siempre son exactamente la misma cosa.

Microsoft documenta tres grandes comportamientos para deployment types en Foundry Models:

- **Global**: prompts y respuestas pueden procesarse en cualquier región donde esté desplegado el modelo.
- **Data Zone**: prompts y respuestas se procesan dentro de la zona de datos especificada por Microsoft, por ejemplo **UE** o **EE. UU.**
- **Standard/Regional**: prompts y respuestas se procesan en la **región de deployment**.

This toolbar appears only when signed in to GitBook.

Los datos en reposo permanecen en la geografía asignada del recurso para todos los tipos, pero el procesamiento ir

Curso Azure AI
0 minutes ago

Segundo principio: identidad fuerte antes que secretos compartidos

Azure OpenAI y Foundry soportan autenticación con **Microsoft Entra ID** y, en algunos escenarios, API keys. Para entornos productivos y sensibles, la ruta recomendada es identidad corporativa: Entra ID, **managed identities** y permisos mínimos.

La documentación de planning de Foundry recomienda configurar la autenticación de conexiones con **Entra ID managed identities** o con **Entra ID passthrough** cuando necesitas mayor control al acceder a fuentes sensibles. La arquitectura de referencia de chat con Foundry muestra además el patrón donde **managed identities** conectan Foundry Agent Service con el proyecto Foundry y éste con Azure OpenAI.

Tercer principio: aislamiento de red real, no solo "cerrar el portal"

Microsoft documenta el aislamiento de red de Foundry alrededor de tres áreas:

- acceso **inbound** al recurso/proyecto,
- acceso **outbound** desde el recurso,
- y acceso **outbound** del cliente de Agent Service hacia dependencias privadas o endpoints aprobados mediante **virtual network injection**. También indica que los recursos PaaS como proyecto Foundry, Storage, Key Vault, Container Registry y Monitoring pueden aislarse con **Private Link**.

Además, Foundry Agent Service ofrece un **Standard Setup with private networking** donde tú aportas una **VNet privada** para crear un entorno aislado con control completo de la infraestructura de red.

Cuarto principio: cifrado con control de claves cuando el requisito lo exige

Los datos en reposo están cifrados con **AES-256**. Para controles más estrictos, Foundry permite usar **Customer-Managed Keys (CMK)** almacenadas en **Azure Key Vault**, añadiendo una capa extra de protección y ayudando a cumplir requisitos de compliance. La documentación indica que el CMK aplica a datos en reposo de cuentas de almacenamiento asociadas al recurso Foundry, incluidos **artefactos de proyecto, archivos cargados y datos de evaluación**.

Requisitos prácticos:

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

- Key Vault y Foundry
- el vault debe tener **soft delete** y **purge protection** habilitados,

- y la identidad administrada usada para el recurso necesita permisos como **Key Vault Crypto User** si usas Azure RBAC. Además, la configuración recomendada con red privada es Key Vault con **private endpoint** y **Allow trusted Microsoft services** habilitado.

Quinto principio: minimizar dato y separar lo que va a memoria, lo que va a contexto y lo que va a knowledge

Los modelos son **stateless**, lo que ayuda a recordar una idea importante: el sistema no "recuerda" por sí mismo; lo que recuerde o conserve depende de tu arquitectura.

Por eso, en entorno regulado, conviene ser muy cuidadosocon:

- qué datos pasan al prompt,
- qué datos persisten,
- qué datos van a una base de conocimiento,
- y qué datos deben quedar fuera del flujo .

En práctica, esto se traduce en:

- **minimización del contexto**,
- **separación por dominio, cliente o unidad**,
- y evitar que el agente vea más de lo estrictamente necesario para cada tarea.

Conceptos clave

> **Customer-Managed Keys**
>
> Claves controladas por el cliente, integradas con Azure Key Vault, para reforzar cifrado en reposo y alinearse con políticas de compliance.

Uso responsable con filtros, límites y decisiones de despliegue alineadas al riesgo

> **Uso responsable = fil ar de forma coherente con el riesgo del dat**

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

En un sistema agéntico, el riesgo no está solo en que el modelo “diga algo inapropiado”. También está en que:

- siga una instrucción maliciosa,
- use una tool cuando no debe,
- vea más datos de los necesarios,
- o procese información en un alcance geográfico que no cumple los requisitos del cliente.

Microsoft Foundry y Azure OpenAI abordan esto con una combinación de **content filtering**, **Prompt Shields**, **guardrails**, tipos de deployment con distinto comportamiento de residencia y procesamiento, y controles de red e identidad.

Los guardrails en Foundry, por ejemplo, se definen como controles que especifican **qué riesgo se detecta, en qué punto se inspecciona y qué acción se toma**; en el caso de agentes, esos guardrails siguen en **preview**.

> El uso responsable no es una feature aislada.
> Es una **estrategia de control** distribuida por toda la arquitectura. (

Filtros base: qué inspeccionan y qué puedes configurar

El sistema de **content filtering** corre junto a los modelos y evalúa tanto el **prompt** como la **completion** con modelos de clasificación diseñados para detectar contenido dañino. Ese filtrado cubre cuatro categorías principales: **violence, hate, sexual y self-harm**.

Estas cuatro categorías se evalúan por **niveles de severidad**. En la configuración de filtros, los umbrales configurables para prompts y completions son **low, medium y high**; el nivel **safe** se anota, pero no se filtra ni es configurable.

Además de las cuatro categorías principales, las respuestas de filtrado pueden incluir señales sobre **jailbreak**, **profanity**, **customer blocklist**, **protected material text** y **protected material code**. Eso es importante porque te permite no solo bloquear, sino también **anotar y enrutar** decisiones dentro de tu aplicación.

Prompt Shields y guardrails: proteger el contexto, no solo la salida

Microsoft documenta **Pro**[illegible]s de manipular el comportamiento del mod[illegible]entación distingue dos ataques principales:

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

- **user prompt attacks**, donde el usuario intenta saltarse las instrucciones del sistema,
- y **document attacks**, donde instrucciones ocultas en documentos, correos o páginas web intentan secuestrar la sesión del modelo.

Los **agent guardrails** en Foundry están en **preview**, así que no conviene apoyarse solo en ellos en escenarios muy regulados sin compensaciones adicionales.

Concepto	Qué es realmente	Dónde se configura / usa
Content Filtering	Sistema integrado de filtrado de contenido para modelos Azure OpenAI / Foundry Models. Está basado en **Azure AI Content Safety**.	Foundry Portal / configuración de content filters / deployment. También devuelve anotaciones en respuestas de la API del modelo.
Prompt Shields	Detector de ataques de prompt injection y document injection. Forma parte del sistema de filtros/guardrails.	Foundry Guardrails, content filters, o llamada API según escenario.
Guardrails	Abstracción de Foundry: colección de controles de seguridad aplicables a modelos y agentes.	Foundry Portal o REST API. Se asigna a deployments o agents.

Uso responsable por nivel de riesgo

Riesgo bajo

- filtros base activos,
- thresholds medios o altos según el caso,
- deployment Global o Data Zone si no hay restricción fuerte,
- logging y observabilidad normales.

Riesgo medio

- filtros ajustados por categoría y por input/output,
- Prompt Shields activados,
- límites estrictos de longitud y formato,
- Data Zone o Regional si ya importa residencia,
- revisión humana en flujos sensibles.

Riesgo alto o regulado

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

- Regional deployment,

- identity fuerte y red privada,
- CMK si el cliente lo exige,
- Prompt Shields + Content Safety,
- salida estructurada y límites estrictos,
- minimización de contexto,
- y evitar depender solo de features preview como única barrera de control.

Integración con otros servicios de Azure para construir soluciones completas

Azure AI Search: grounding, RAG y conocimiento corporativo

Azure AI Search es probablemente la integración más natural con Azure OpenAI cuando el problema exige **responder con contexto corporativo**. Azure AI Search es un servicio para **RAG** con recuperación híbrida y agentic retrieval, y señala que puede convivir con código de orquestación propio cuando necesitas control fino del pipeline. Una solución con agentes y Azure AI Search puede beneficiarse de **Foundry IQ** como endpoint único de conocimiento.

Azure Key Vault: secretos, claves y separación de responsabilidades

Key Vault aparece cuando la solución deja de ser un experimento y necesita una gestión seria de:

- secrets,
- credentials,
- certificates,
- y claves de cifrado.

Key Vault reduce el riesgo de fuga accidental porque evitas almacenar información de seguridad en la aplicación.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Azure Monitor y Application Insights: observabilidad real

Se propone **Application Insights** como APM integrado con **OpenTelemetry** y, específicamente para IA generativa, Foundry Observability se integra con Azure Monitor Application Insights para mostrar métricas de rendimiento, seguridad y calidad. Además, la vista de agentes de Application Insights permite monitorizar agentes procedentes de Foundry, Copilot Studio y otras fuentes.

Azure Functions y Logic Apps: tools y workflows que actúan de verdad

Azure Functions como compute serverless y como base para tools o MCP servers. Además, Foundry ofrece tools específicas de Azure Functions con comunicación basada en colas para **function calling asíncrono** mediante patrones de mensajería.

Azure Logic Apps como workflows de integración multistep con más de **1.400 conectores**, invocables desde agentes de Foundry para automatizar tareas de negocio y coordinar sistemas Microsoft y no Microsoft.

API Management: gobernanza, seguridad y control del tráfico de IA

Cuando la solución crece, aparece una necesidad nueva: **no solo integrar APIs, sino gobernarlas**.

AI gateway de Azure API Management es un conjunto de capacidades para **secure, scale, monitor and govern** endpoints de IA, incluyendo Azure OpenAI y Microsoft Foundry. También documenta autenticación y autorización hacia APIs de IA mediante **API key** o **Microsoft Entra managed identity** desde APIM.

Hosting: App Service y Container Apps

Una solución completa necesita **vivir en algún sitio**.

Microsoft documenta **Azure App Service** como plataforma adecuada para chatbots, aplicaciones RAG y también agentic web apps que integran Azure OpenAI, Azure AI Search, LangGraph o Foundry Agent Service. App Service aparece con managed identities para conectar de forma passw

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Microsoft también documenta **Azure Container Apps** como opción para entornos containerizados con integración con Foundry, útil cuando necesitas más control de runtime o empaquetado de servicios.

Event Grid y Service Bus: cuando la solución es asíncrona o event-driven

Microsoft documenta **Azure Event Grid** como servicio publish-subscribe altamente escalable para arquitecturas orientadas a eventos, y **Azure Service Bus** como message broker empresarial para colas y pub/sub duradero, útil para desacoplar aplicaciones y coordinar trabajo con alta fiabilidad.

> **Modelo para inferir.**
> **Search para fundamentar.**
> **Functions/Logic Apps para actuar.**
> **Key Vault para proteger.**
> **Monitor para observar.**
> **APIM para gobernar.**
> **App Service/Container Apps para servir.**
> **Event Grid/Service Bus para desacoplar.**

Ejemplo: Azure AI Search + Azure OpenAI

```
import os
from azure.identity import DefaultAzureCredential
from azure.search.documents import SearchClient
from openai import OpenAI

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default"
)

# Cliente de búsqueda
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: patrón asíncrono con Service Bus

```
import os
from azure.identity import DefaultAzureCredential
from azure.servicebus import ServiceBusClient, ServiceBusMessage

servicebus_client = ServiceBusClient(
    fully_qualified_namespace=os.environ["SERVICEBUS_NAMESPACE"],
    credential=DefaultAzureCredential()
)

queue_name = os.environ["SERVICEBUS_QUEUE"]
```

Buenas prácticas para elegir bien modelo, conguración y estrategia de consumo

Conviene diferenciar entre modelos orientados a **reasoning profundo**, como GPT-5, y modelos orientados a **respuesta rápida y alto throughput**, como GPT-4.1.

Elegir bien el modelo: primero la tarea, luego el catálogo

La primera buena práctica es no elegir modelo “por marca” o por prestigio, sino por **tipo de tarea**. Por ejemplo GPT-4.1 para asistentes de atención, soporte y workloads sensibles a latencia, y GPT-5 para casos de razonamiento estructurado, planificación y orquestación más compleja.

Además, GPT-4.1 ofrece mayor throughput y menor latencia, mientras que GPT-5 ofrece más profundidad analítica a costa de mayor tiempo de respuesta.

Una regla práctica útil para clase:

- **chat en vivo, soporte, FAQ, Q&A corto** → modelo rápido
- **planificación, análisis, tool calling complejo, research** → modelo de reasoning
- **RAG y recuperación semántica** → embeddings, no chat
- **clasificación o routing** → variantes ligeras o modelos más económicos, si la calidad sigue siendo suficiente

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

No uses el mismo modelo para todo

Otra buena práctica crítica es **separar responsabilidades por deployment**.

Configuración de inferencia: empieza por controlar estructura y presupuesto

1. **Primero controla longitud** con `max_tokens` o su equivalente.
2. **Después controla formato** con `response_format`, especialmente si el agente alimenta tools o sistemas.
3. **Luego ajusta variabilidad** con `temperature` o `top_p`.
4. **Solo después** afina creativity o estilo.

En agentes, la estructura suele importar más que la creatividad

`response_format` soporta `json_object` y `json_schema`, y además indica que para modelos que lo soportan **`json_schema`** **es preferible** al antiguo JSON mode.

Si usas `json_object`, debes instruir explícitamente al modelo para producir JSON; si no, puede generar una salida problemática o incluso parecer “atascado” hasta alcanzar el límite de tokens

Reasoning effort: úsalo donde aporta, no como valor por defecto

Reducirlo puede dar **respuestas más rápidas** y usar **menos tokens de razonamiento**.

Diseña con cuota y crecimiento en mente

La cuota de Azure OpenAI se define por **suscripción**, **región**, **modelo o deployment type**, y se expresa en **TPM** y **RPM**.

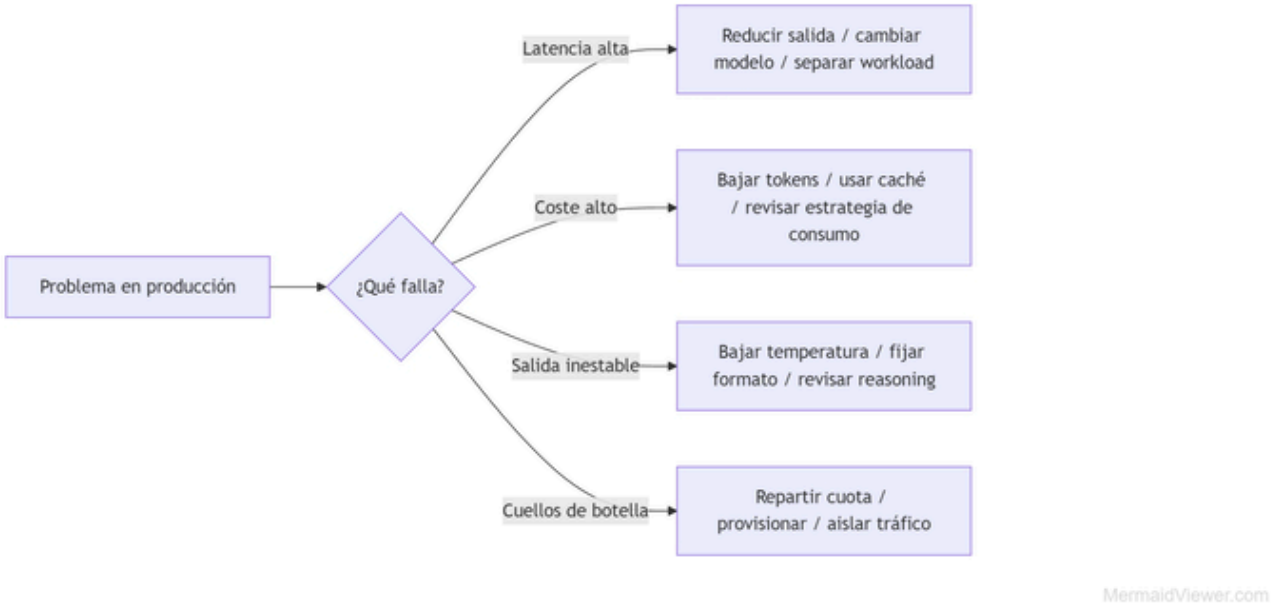
Ahora existen **Quota Tiers** que aumentan automáticamente con el uso, pero la arquitectura sigue teniendo que decidir cómo reparte esa cuota entre deployments.

Además, con suficiente cuota regional, puedes repartir TPM entre varios deployments dentro de la misma región o incluso entre varios recursos, siempre que la suma respete la cuota disponible.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Qué tocar primero



MICROSOFT AGENT FRAMEWORK (MAF)

Qué es Microsoft Agent Framework y qué lugar ocupa en el stack de agentes de Microsoft

Microsoft Agent Framework es el framework actual de Microsoft para construir agentes de IA y workflows multiagente en .NET y Python.

Es un marco para desarrollar **agents** y **workflows**: los agentes procesan entradas con LLMs, llaman tools o servidores MCP y generan respuestas; los workflows conectan agentes y funciones para tareas multi-step con **ruteo tipado**, **checkpointing** y **human-in-the-loop**.

MAF vive en la **capa de framework**, no en la capa de modelo ni en la capa de plataforma gestionada. **Semantic Kernel** y **AutoGen** fueron pioneros en los conceptos de agentes y orquestación multiagente, y Agent Framework es su **sucesor directo**, creado por los mismos equipos.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Capacidades Principales

1. Agents

Los **agents** son componentes que usan LLMs para procesar entradas, llamar tools y generar respuestas. El framework soporta distintos tipos de agente y todos derivan de una base común, `AIAgent`, lo que facilita construir funcionalidad agnóstica al tipo de agente y patrones de orquestación más reutilizables.

Soporta múltiples proveedores de inferencia, no solo Azure: la documentación menciona Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama y cualquier servicio que implemente el contrato esperado del chat client.

2. Workflows

Los **workflows** son la pieza diferenciadora cuando el proceso tiene pasos bien definidos o coordinación compleja. Destacan:

- ruteo tipado,
- checkpointing,
- recuperación y reanudación de procesos largos,
- patrones multiagente incorporados,
- y soporte para intervención humana.

También hay patrones de orquestación listos para usar como **sequential**, **concurrent**, **hand-off** y **magentic**.

Es la **capa de framework y orquestación**. Aquí defines agentes, tools, estado, workflows, handoffs y control explícito del proceso.

MAF encaja muy bien cuando:

- necesitas **control del flujo** y no quieres depender solo de la improvisación del LLM,
- quieres **multi-agent orchestration** con patrones claros,
- el sistema necesita **estado**, **telemetría** y **checkpointing**,
- o quieres una base pro-code más robusta que un agente definido solo por configuración.

MAF no debería confundirse con:

This toolbar appears only when signed in to GitBook.

- una **plataforma fully** gent Service, que se encarga de hosting,

Curso Azure AI
0 minutes ago

- una **herramienta de experiencia low-code**: ahí encajan mejor capas como Copilot Studio o interfaces M365, no el framework en sí.

Conceptos clave

Agent

Componente que usa un LLM, contexto y tools para responder o actuar. En MAF, todos los agentes derivan de una base común.

Workflow

Grafo tipado que conecta agentes y funciones para tareas multi-step con control explícito, checkpointing y soporte human-in-the-loop

Type safety

Validación y contratos más robustos entre componentes, especialmente útil en sistemas multi-step y multi-agent. Es una de las capacidades enterprise heredadas de Semantic Kernel.

Human-in-the-loop

Capacidad de introducir intervención humana dentro del flujo cuando el caso lo requiere.

Hosted agent

Tipo de agente que puedes desplegar en Foundry Agent Service usando Agent Framework, LangGraph o tu propio código.

`OpenAIChatCompletionClient` usa **Chat Completions API**, es la opción más simple y con soporte amplio de modelos. Tiene herramientas como function tools, web search y MCP local.

`OpenAIChatClient` usa **R**[illegible] especialmente cuando quieres hosted tools com[illegible]sted MCP, tool approvals y MCP local.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Para opciones mas avanzadas de Azure se puede usar `FoundryChatClient`

Ejemplo: Agente Mínimo con MAF Responses

build_client.py

```
def build_maf_chat_client(
    endpoint: str,
    model: str,
    api_key: Optional[str] = None,
    api_version: str = DEFAULT_API_VERSION,
):
    """
    Crea un OpenAIChatCompletionClient para Microsoft Agent Framework.

    - Si api_key viene informada, usa API Key.
    - Si api_key está vacía, usa Microsoft Entra ID / RBAC.
```

az203.py

```
import asyncio
import os
from pathlib import Path

from dotenv import load_dotenv
from agent_framework import Agent

from build_client import build_maf_responses_client

load_dotenv("../.env")
```

Ejemplo: tool con contrato y validación

```
@tool(approval_mode="never_require")
def get_weather(
    location: Annotated[str, Field(description="Ciudad para consultar el tiempo")]
) -> str:
    # En producción, validarías y sanitizarías aquí el input
    return f"El tiempo en {location} es soleado y 24°C."

#...

agent = Agent(
    client=client,
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Relación con LangGraph como referencia para equipos que ya trabajan con grafos y estado

> **LangGraph y MAF comparten una intuición importante:**
> **cuando el sistema necesita control, estado y rutas explícitas, conviene modelarlo como un grafo o workflow y no como una simple conversación improvisada por el LLM.**

LangGraph suele ser la referencia natural para equipos que vienen del ecosistema open source y piensan en `StateGraph`, nodos y edges; mientras que **MAF** encapsula esa misma necesidad dentro de un framework Microsoft que añade capacidades enterprise como **estado por sesión, type safety, middleware, telemetry** y **workflows basados en grafos con routing tipado, checkpointing y human-in-the-loop**

Los workflows de MAF son **graph-based**, con **type-based routing**, **checkpointing** y soporte para **human-in-the-loop**.

La equivalencia mental más útil

- **Graph / StateGraph** en LangGraph → **Workflow** en MAF
- **Node → Executor, Agent o Function dentro del workflow**
- **Edges / routing → Type-based routing y patrones de orquestación**
- **State → session-based state, context y checkpointing**
- **Human approval / interrupt → human-in-the-loop request/response patterns**

No es una equivalencia 1:1 exacta, pero sí una traducción conceptual muy válida

Dónde se parecen

1. Ambos premian el control explícito

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

LangGraph es popular precisamente cuando el equipo no quiere delegar toda la lógica a un agente “monolítico”. MAF va en la misma dirección: es un framework para construir agentes y workflows con control explícito del proceso, soporte de patrones secuenciales, concurrentes, handoff, group chat y magentic orchestration.

2. Ambos tratan el estado como una pieza de diseño

LangGraph enseña a pensar en estado como parte central del flujo. MAF también: Microsoft destaca **session-based state management** como una de las capacidades enterprise heredadas de Semantic Kernel, y los workflows soportan **checkpointing** para persistir y reanudar procesos largos.

3. Ambos encajan bien cuando el sistema tiene varias etapas o agentes

En LangGraph tiene sentido modelar rutas, delegación y coordinación. En MAF, existen orquestaciones multiagente ya reconocibles para alguien que viene de grafos: sequential, concurrent, handoff, group chat y magentic.

4. Ambos son piezas de framework, no plataformas gestionadas

LangGraph es similar a Microsoft Agent Framework y es un SDK. Esto ayuda mucho en clase porque deja claro que ni LangGraph ni MAF son, por sí mismos, la plataforma operativa; son la capa de construcción y orquestación.

Diferencias importantes

1. MAF entra más claramente en el discurso enterprise de Microsoft

Microsoft posiciona Agent Framework como sucesor de AutoGen y Semantic Kernel, combinando abstracciones simples de agentes con **type safety, middleware, telemetry, state management** y workflows basados en grafos. Ese énfasis en capacidades enterprise está mucho más explícito que en la documentación típica de LangGraph

2. MAF estructura más la orquestación alrededor de workflows tipados

Agent Framework se centra en un **typed, graph-based Workflow** que enruta datos por edges y activa executors cuando las entradas están listas.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: LangGraph + Foundry

```
import os
from azure.identity import DefaultAzureCredential
from langchain_azure_ai.agents import AgentServiceFactory

factory = AgentServiceFactory(
    project_endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)

support_node = factory.get_agent_node(
    name="support-agent",
```

Relación con LangChain como marco más amplio de composición y tooling

Qué papel juega LangChain

LangChain se considera un marco para **encadenar componentes**: modelos, prompts, parsers, retrievers, vector stores, tools, memoria y, en muchos casos, grafos con LangGraph.

`langchain-azure-ai` publica namespaces diferenciados para **Foundry Agent Service**, **chat models**, **embeddings**, **vector stores**, **retrievers** y **Content Safety middleware**.

Diferencias importantes

1. LangChain tiene una superficie de composición más amplia

LangChain sigue siendo especialmente fuerte cuando quieres pensar en la aplicación como **composición de piezas heterogéneas**: modelos, embeddings, retrievers, vector stores, middleware, tracing y agentes.

2. MAF concentra más el discurso en agente + workflow

La propuesta de MAF está más cerrada alrededor de dos ejes:

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

- **Agents** como unidad
- **Workflows** como mec

Patrón: LangChain para composición, Foundry para capacidades, MAF cuando el workflow lo pida

La documentación actual permite defender un patrón híbrido muy razonable:

- **LangChain** como marco amplio de composición,
- **Foundry** como proveedor de capacidades gestionadas,
- y **MAF** cuando el caso evoluciona hacia workflows agénticos más estrictos.

Conceptos clave

Framework agéntico

Enfoque donde el centro de gravedad está en el agente, sus tools, su estado y sus workflows. MAF se presenta así de forma explícita.

`langchain-azure-ai`

Paquete oficial documentado por Microsoft como punto de entrada para LangChain y LangGraph con capacidades de Foundry.

`AgentServiceFactory`

Clase documentada por Microsoft para crear nodos compatibles con LangGraph que se ejecutan mediante Foundry Agent Service. Aunque aparece sobre todo en el contexto de LangGraph, también refuerza la continuidad entre LangChain/LangGraph y el stack Microsoft.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: LangChain como composición de modelo + prompt (sin MAF)

az204.py

```
import os

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv("../.env")

endpoint = os.getenv("AOAI_ENDPOINT_SECONDARY")
api_key = os.getenv("AOAI_API_KEY_SECONDARY")
```

Abstracciones principales para diseñar agentes con componentes bien definidos

> **Un agente bien diseñado no es una única caja negra.**
> **Es una composición de abstracciones con responsabilidades distintas.**

Esas abstracciones caen en dos categorías principales, **Agents** y **Workflows**, y varios bloques fundacionales: **model clients**, **agent session** para state management, **context providers** para memoria y contexto, **middleware** para interceptar acciones y **MCP clients** para integración de herramientas.

Primera abstracción: el model client

MAF separa el acceso al modelo del comportamiento del agente.

- desacopla el agente del proveedor concreto,
- permite cambiar backend de inferencia sin reescribir toda la lógica,
- separa claramente “quién genera” de “quién decide cómo usar esa generación”.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Segunda abstracción: el agente como unidad de comportamiento

Se definen los **agents** como componentes que usan LLMs para procesar entradas, llamar **tools** y **MCP servers**, y generar respuestas.

`BaseAgent` se describe como clase base con soporte para **context providers, middleware y thread management**, y `ChatAgent` como la implementación principal basada en un chat client, con soporte para tools y respuestas streaming y no streaming.

- encapsula instrucciones, herramientas y comportamiento,
- convierte el LLM en un componente operativo más controlado,
- y permite reutilización del mismo patrón de agente con distintos providers.

Tercera abstracción: tools y MCP como frontera de acción

En MAF, el agente puede llamar **tools** y **MCP servers**.

- separa razonamiento de ejecución,
- aisla side effects,
- pone contratos claros en las acciones externas,
- y permite auditoría de qué hizo realmente el sistema.

Cuarta abstracción: context providers y memoria contextual

MAF introduce **context providers** como pieza fundacional. Es un mecanismo para memoria y contexto del agente

- Separa la memoria y grounding del prompt,
- distingue entre contexto que entra desde fuera y lógica interna del agente,
- y controla mejor qué información se recupera, cuándo y con qué riesgo.

Quinta abstracción: agent session y state management

Los agentes gestionan contexto mediante `AgentSession`, y el orquestador sincroniza ese contexto cuando participa

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

- permite continuidad
- separa el estado por sesión o usuario,

- soporta procesos largos,
- y es una base para patrones multiagent sin mezclar todo en variables globales

Sexta abstracción: middleware como capa transversal

El middleware de chat permite interceptar y modificar llamadas al chat client, por ejemplo para logging, transformación de prompts o transformación de respuestas.

- logging y seguridad transversales,
- inyección de contexto dinámico,
- políticas de validación,
- y transformación de llamadas sin contaminar la lógica principal del agente.

Séptima abstracción: workflow como control explícito del proceso

Se definen los **workflows** como grafos que conectan agentes y funciones para tareas multi-step con **type-safe routing, checkpointing y human-in-the-loop**.

Un workflow es distinto de un agent: el agent actúa de forma más dinámica guiado por el LLM, mientras que el workflow representa una secuencia o red de operaciones **predefinida**, con control explícito del flujo.

Un workflow une **executors** y **edges** en un directed graph, coordina invocation, routing y event streaming, y que se ejecuta con un modelo tipo Pregel/supersteps.

- controla rutas y decisiones,
- compone pasos deterministas y agénticos,
- reanuda procesos largos,
- y separa “qué hace cada componente” y “cómo se coordina todo”.

Octava abstracción: executor como unidad mínima del workflow

Los **executors** son los bloques fundamentales que procesan mensajes en un workflow.

Son unidades autónomas que reciben mensajes tipados, ejecutan operaciones y producen mensajes o eventos. Puec

This toolbar appears only when signed in to GitBook.

`AgentExecutor` es el ad

Curso Azure AI
0 minutes ago

or de workflow pueda enrutarle mensajes, gestionar su sesión y manejar su salida.

- convierte cualquier paso en componente interoperable,
- mezcla agentes y funciones dentro del mismo workflow,
- y mantiene contratos claros entre nodos del proceso.

Estado, memoria y tools como piezas clave del comportamiento del agente

> **El modelo razona sobre el contexto disponible.**
> **El estado decide qué continuidad existe.**
> **La memoria decide qué merece persistir o recuperarse.**
> **Y las tools deciden hasta dónde puede actuar el agente.**

AgentSession es la abstracción central de conversación y estado.

Hay un patrón muy claro: crear una sesión, pasarla a cada `run(...)`, serializarla, restaurarla y continuar la conversación cuando haga falta.

Las sesiones son **específicas del agente o del servicio**; reutilizarlas con otra configuración o con otro provider puede producir contexto inválido.

Cuando un workflow usa agentes como ejecutores, se crea una sesión para gestionar el estado de la conversación, y que el **AgentExecutor** puede serializar ese estado como parte del checkpoint.

Memoria: qué contexto merece persistir o recuperarse

1. Historial conversacional

Los agentes guardan historial de chat en un `InMemoryHistoryProvider` o en el servicio subyacente, según lo que requiera ese proveedor de IA.

Al crear una sesion, se almacenará historial. Adema se pueden almacenar hechos concretos con session.state["variable"]

2. Memoria útil o contextual

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Los **context providers** son componentes que participan en la pipeline del agente, pueden **añadir o sobreescribir mensajes de entrada** antes de la invocación, y pueden **extraer información** después de la ejecución.

Esos providers sirven tanto para **memoria** como para **RAG**. Por ejemplo, existen providers de historial, de memoria y de búsqueda textual

Esto da una regla didáctica muy potente:

- **estado** es continuidad de la ejecución o sesión,
- **memoria** es contexto recuperable o persistible que puede enriquecer decisiones futuras
- y **prompt** no debería ser el único lugar donde vive esa información

En Python puede haber varios history providers, pero solo uno debería usar `load_messages=True`, para evitar conflictos en la fuente principal de historial.

Tools: la frontera entre razonar y actuar

Las tools son parte del comportamiento normal del agente: los agents procesan inputs, llaman **tools** o **MCP servers**, y generan respuestas. Hay dos tipos:

- **function tools**, con funciones locales o de aplicación,
- **MCP tools**, basadas en Model Context Protocol, un estándar abierto para exponer tools y datos contextuales a los LLMs.

Conceptos clave

> **AgentSession**
>
> Es la unidad de continuidad conversacional o de ejecución. Permite crear, serializar, restaurar y continuar conversaciones, y es específica del agente o del servicio usado.

> **Context provider**
>
> Es un componente que participa antes y después de cada invocación para añadir contexto, modificar entradas o e memoria y de RAG en MAF.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Chat history provider

Es un tipo especializado de provider para cargar o almacenar historial conversacional. Por defecto puede ser en memoria o apoyarse en el servicio subyacente, según el provider.

Tool

Es una acción externa invocable por el agente. Puede ser una función local o remota, pero debe tratarse como frontera de confianza y tener contrato claro.

MCP

Model Context Protocol es un estándar abierto para proporcionar tools y datos contextuales a los LLMs. Agent Framework soporta integración con MCP servers

Checkpointing

Permite persistir y reanudar ejecuciones largas; en agentes y workflows incluye la serialización del estado de sesión.

Ejemplo: estado de sesión entre turnos

az205.py

```
import asyncio
import os
from pathlib import Path

from dotenv import load_dotenv
from agent_framework import Agent

from build_client import build_maf_responses_client

load_dotenv("../.env")
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejercicio. Chatbot Soporte

Diseñar un miniagente/chatbot que:

- Lea input del usuario hasta que el usuario quiera salir
- Mantenga sesion entre interacciones

Ejemplo: memoria conversacional local mediante custom provider

```
import asyncio
from agent_framework import Agent, InMemoryHistoryProvider
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

from collections.abc import Sequence
from typing import Any

from agent_framework import BaseHistoryProvider, Message

class DatabaseHistoryProvider(BaseHistoryProvider):
```

Type safety y contratos para construir integraciones más robustas y mantenibles

En MAF, el valor de **type safety** aparece en varios niveles: en los **workflows**, donde los executors reciben y producen mensajes tipados; en las **tools**, donde los parámetros y esquemas pueden definirse explícitamente; y en las **salidas estructuradas**, donde `response_format` puede obligar al modelo a devolver JSON parseable para un modelo de datos concreto.

Contratos en tools: la primera frontera que conviene formalizar

Cualquier función Python puede convertirse en una function tool y que, para describir mejor su contrato, puedes usar `Annotated` y `Pydantic Field`.

Cuando necesitas control total del esquema expuesto al modelo, `@tool` admite un `schema` basado en **Pydantic** o en

This toolbar appears only when signed in to GitBook.

Además, la clase `AIFun` función invocable por modelos con **automatic parameter validation** y **JSON schema generation**.

Curso Azure AI
0 minutes ago

Contratos en salidas: structured outputs en vez de texto ambiguo

Usar **Pydantic models** como contrato de salida y pasar ese modelo a `response_format`, de forma que el LLM deba devolver JSON parseable. `response_format` fuerza que la salida sea JSON interpretable por el modelo Pydantic, y luego los edges pueden inspeccionar esa estructura para tomar decisiones de enrutado.

Type safety en workflows: mensajes tipados y rutas más fiables

Los **executors** son los bloques fundamentales del workflow y reciben **typed messages**, ejecutan operaciones y producen mensajes o eventos.

Type safety no sustituye a la validación de seguridad.

El modelo puede llamar cualquier función que le proporciones como tool y elegir los argumentos.

Se recomienda tratar los argumentos propuestos por el LLM como **untrusted input**, igual que input de usuario en una web API.

Además, se clasifica los roles de mensajes por nivel de confianza:

- `system` con el nivel más alto,
- `user`, `assistant` y `tool` como no confiables.

Middleware como lugar natural para reforzar contratos

El **chat-level middleware** es una capa capaz de interceptar y modificar llamadas al chat client subyacente. Se propone para logging, modificación de prompts antes de llegar al servicio de IA o transformación de respuestas.

Eso lo convierte en un punto muy razonable para aplicar políticas de contrato transversales, por ejemplo:

- validar que ciertas respuestas estructuradas existen,
- rechazar invocaciones incompletas,
- normalizar campos,
- o registrar violacione s capas.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Conceptos clave

> **AIFunction**
>
> Wrapper de una función Python para hacerla invocable por modelos con **automatic parameter validation** y **JSON schema generation**

Ejemplo: schema explícito con Pydantic

La documentación oficial permite pasar un modelo Pydantic o un JSON schema explícito a `@tool(schema=...)`.

```
from typing import Annotated
from pydantic import BaseModel, Field
from agent_framework import tool


class CreateTicketInput(BaseModel):
    project: Annotated[str, Field(description="Proyecto afectado")]
    priority: Annotated[str, Field(description="Prioridad: baja, media o alta")]
    summary: Annotated[str, Field(description="Resumen corto de la incidencia")]
```

Ejercicio. Modelo Pydantic

- Añadir una herramienta al chatbot, de forma que cuando el usuario quiera abrir incidencia, se invoque y devuelva un numero de ticket

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: contexto de ejecución oculto al modelo

```
from typing import Annotated
from pydantic import Field
from agent_framework import tool, FunctionInvocationContext


@tool
def get_customer_status(
    customer_id: Annotated[str, Field(description="Identificador del cliente")],
    ctx: FunctionInvocationContext,
) -> str:
    # Valor inyectado en runtime, no elegido por el modelo
```

Propiedad	Uso
`ctx.function`	Tool/función que se está invocando. Útil para logging o políticas por tool.
`ctx.arguments`	Argumentos ya validados de la tool. Normalmente un modelo Pydantic.
`ctx.kwargs`	Datos de runtime pasados desde `agent.run(..., function_invocation_kwargs={...})`.
`ctx.metadata`	Diccionario para compartir datos entre middlewares.
`ctx.result`	Resultado de la función. Se puede inspeccionar o sobrescribir.
`ctx.terminate`	Si se pone a `True`, corta la ejecución posterior del pipeline.

Orquestación y workows para coordinar tareas, rutas y decisiones con mayor control

En cada paso de un proceso puedes decidir si **debe decidir el modelo**, **debe decidir el código** o **debe decidir un humano**.

Un workflow en Agent Framework ejecuta un **grafo dirigido** de **executors** conectados por **edge groups**, usando un modelo en **supersteps** hasta que el grafo queda inactivo.

Resume sus conceptos principales en cuatro piezas:

- **Executors**: unidades d
- **Edges**: conexiones y
- **Events**: señales de ob

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

- **Workflow Builder & Execution**: mecanismo que une todo y gestiona la ejecución.

Executors: las unidades mínimas del proceso

Los **executors** son los bloques fundamentales que procesan mensajes dentro de un workflow. Son unidades autónomas que reciben **typed messages**, realizan operaciones y pueden producir nuevos mensajes o eventos. Además, los executors pueden ser dos cosas:

- **custom logic components**, que transforman datos, llaman APIs o implementan lógica **determinista**,
- **AI agents**, cuando lo que necesitas es razonamiento guiado por un LLM.

AgentExecutor: cómo entra un agente dentro del workflow

Agent Executor es la pieza que adapta un agente al modelo de ejecución de workflows. Su trabajo es:

- recibir mensajes tipados desde el grafo,
- reenviarlos al agente,
- gestionar la sesión y el estado conversacional del agente,
- adaptarse a ejecución streaming o no streaming,
- emitir eventos de salida observables,
- enviar mensajes a executors downstream,
- y soportar checkpointing en workflows largos.

> El workflow no habla directamente “con el LLM”.
> Habla con **executors**, y el agente entra en ese mundo a través de **AgentExecutor**.

Edges: cómo se modelan rutas y decisiones

Los **edges** definen cómo fluyen los mensajes entre executors y que pueden incluir condiciones basadas en el contenido del mensaje. Además, enumera cinco patrones de edge soportados:

- **Direct**: conexión lineal uno a uno,
- **Conditional**: if/else se
- **Switch-Case**: varias
- **Multi-Selection (Fan-**

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

- **Fan-in**: varios nodos convergen en uno.

Estado compartido: coordinación más allá del paso de mensajes

El **state** en workflows permite que varios executors accedan y modifiquen datos comunes cuando el paso directo de mensajes no es suficiente o no es eficiente.

Checkpointing

Es un mecanismo para guardar estados y reanudar procesos largos, incluyendo:

- **estado local de cada executor**,
- **estado compartido entre executors**,
- **colas de mensajes pendientes**,
- y **posición actual del workflow**.

Human-in-the-loop: control humano en puntos explícitos

Se implementa mediante un mecanismo de **request and response handling**, donde los executors pueden enviar peticiones fuera del workflow, por ejemplo a un humano, y esperar la respuesta antes de continuar. La pieza técnica central para esto es `RequestPort`.

Las orquestaciones también soportan aprobación humana mediante **tool approval** o **request info**, lo que refuerza una idea muy importante: la intervención humana no es un “hack” externo, sino una parte modelable del flujo.

Patrones de orquestación incorporados

Hay varios patrones multiagente preconstruidos en Agent Framework:

- **Sequential**: agentes uno detrás de otro,
- **Concurrent**: agentes en paralelo,
- **Handoff**: un agente cede el control a otro según contexto,
- **Group Chat**: varios agentes colaboran en una conversación compartida,
- **Magentic**: un manage orma dinámica.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Ejemplo: patrón secuencial simple

az206.py

```
import asyncio
import os
from pathlib import Path
from typing import cast

from dotenv import load_dotenv
from agent_framework import Agent, Message
from agent_framework.orchestrations import SequentialBuilder

from build_client import build_maf_responses_client
```

Ejercicio. Debug

Mostrar en pantalla todos los eventos que produce el orquestador

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

`event.type` habitual	Qué significa	Cuándo lo verás
`output`	Salida final del workflow	Al terminar correctamente
`request_info`	El workflow pide información externa	Tool approval o pausa para feedback humano
`agent_response` / similar	Un agente ha producido una respuesta completa	Cuando un participante termina su turno
`agent_response_update` / similar	Fragmento de respuesta en streaming	Si el cliente/modelo/orquestador emite streaming
`executor_invoked`	Un agente/executor empieza a procesar	Al entrar en cada participante
`executor_completed`	Un agente/executor termina correctamente	Al salir de cada participante
`executor_failed`	Un agente/executor falla	Error en un participante
`workflow_started`	Empieza el workflow	Inicio de ejecución
`workflow_error`	Error del workflow	Fallo general
`workflow_warning`	Warning no fatal	Situaciones recuperables
`superstep_started`	Empieza una fase/superstep	Más visible en workflows con pasos internos
`superstep_completed`	Termina una fase/superstep	Idem

Observabilidad integrada para seguir ejecuciones, errores y desempeño real

En MAF, la observabilidad se presenta como capacidad integrada del framework, con emisión automática de spans y métricas, y en workflows como un sistema de **eventos** que expone el progreso de la ejecución en tiempo real.

Foundry, por su parte, permite visualizar trazas en el portal y almacenarlas en **Azure Monitor Application Insights** para análisis posterior.

En paralelo, Application Insights ofrece vistas como **Failures**, **Performance**, **Logs**, **Alerts** y una vista unificada de **Agents** dio y terceros.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Microsoft Agent Framework integra **OpenTelemetry** y emite telemetría de acuerdo con las **OpenTelemetry GenAI Semantic Conventions**. Eso significa que el framework no solo permite logs manuales, sino que ya genera una base estándar de **trazas, logs y métricas** sobre las operaciones del agente.

MAF muestra varias formas de activarla:

- por variables de entorno con `configure_otel_providers()`,
- con exporters OTLP personalizados,
- o integrándola directamente con **Azure Monitor** usando el paquete `azure-monitor-opentelemetry`.

Qué se puede observar en un agente

La documentación oficial de MAF describe varios spans:

- `invoke_agent <agent_name>` como span raíz de una invocación,
- `chat <model_name>` cuando el agente llama al modelo,
- `execute_tool <function_name>` cuando ejecuta una function tool

Y también expone métricas automáticas como:

- `gen_ai.client.operation.duration` para duración de operaciones,
- `gen_ai.client.token.usage` para uso de tokens.

Observabilidad específica de workflows

En workflows, Microsoft añade un sistema de **eventos** orientado a ejecución real. La documentación dice que el workflow event system proporciona observabilidad sobre la ejecución y que los eventos se emiten en puntos clave y pueden consumirse **en tiempo real por streaming**.

Entre los tipos built-in documentados aparecen:

- `started`, `status`, `output`, `failed`, `error`, `warning`,
- `executor_invoked`, `executor_completed`, `executor_failed`,
- `superstep_started`, `superstep_completed`,
- `request_info` para solicitudes externas o de aprobación,
- y eventos relacionados [illegible]tes.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Además, existen **custom events** para que los executors emitan señales propias, por ejemplo:

- progreso de una tarea,
- diagnósticos,
- métricas,
- o datos de dominio como escrituras en base de datos o tool calls.

Cómo encaja Foundry

Microsoft Foundry ofrece trazabilidad integrada para frameworks populares y documenta que el tracing captura **LLM calls, tool invocations y agent decision flows** para ayudar a depurar problemas, monitorizar latencia y entender comportamiento.

Para ver trazas necesitas un **Foundry project** conectado a **Azure Monitor Application Insights**, y permisos adecuados sobre el recurso de Application Insights y, en algunos casos, sobre Log Analytics. Una vez configurado, las trazas se visualizan en **Observability > Traces** en Foundry y también pueden explorarse en Azure Monitor.

La vista general de Foundry Observability añade otro matiz importante: su dashboard, integrado con Application Insights, ofrece visibilidad sobre **performance**, **safety** y **quality**.

Application Insights como backend de observabilidad

Application Insights es la pieza de Azure Monitor que Microsoft posiciona como APM para aplicaciones instrumentadas con OpenTelemetry. La documentación oficial resalta varias experiencias especialmente útiles para agentes:

- **Failures view** para fallos,
- **Performance view** para cuellos de botella,
- **Logs** para análisis KQL,
- **Alerts** para alertado,
- **Workbooks** y **Grafana dashboards** para visualización,
- y **Agents details** como vista unificada para agentes de Foundry, Copilot Studio y terceros.

Se puede habilitar un dashboard específico de **Azure Managed Grafana** para workflows de MAF, apoyado en Application Insights y OpenTelemetry. Eso sirve para enseñar que la observabilidad no se limita [illegible] ficos de operación.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Datos sensibles: lo que puedes ver y el riesgo que introduces

- `EnableSensitiveData` permite capturar prompts, respuestas, argumentos de function calls y resultados,
- pero **solo debería activarse en desarrollo o test**,
- porque en producción puede exponer información sensible en logs y traces.

Conceptos clave

OpenTelemetry

Estándar abierto y neutral para instrumentar aplicaciones y emitir trazas, logs y métricas. MAF lo usa como base de su observabilidad integrada

Trace / Span

Una **trace** representa una ejecución distribuida; un **span** representa una operación concreta dentro de esa ejecución. En MAF aparecen spans como `invoke_agent`, `chat` y `execute_tool`.

Application Insights

Servicio APM de Azure Monitor para almacenar y analizar telemetría de aplicaciones, con vistas de fallos, rendimiento, logs, agentes y alertas.

Foundry Traces

Vista de observabilidad en Foundry para seguir trazas de agentes y depurar comportamiento, integrada con Application Insights.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Enfoque enterprise orientado a seguridad, gobierno y operación a escala

Todo agente debe ser **observable, governed and secure** porque los agentes acceden a datos, toman acciones y operan con autoridad delegada.

El sistema debe soportar preguntas como estas:

- ¿quién es responsable de este agente?
- ¿qué permisos tiene?
- ¿qué datos ve?
- ¿qué tools puede ejecutar?
- ¿cómo se audita lo que hace?
- ¿cómo se aísla de otros entornos o clientes?
- ¿cómo se revoca o corrige si algo sale mal?

Identidad y autorización: la base del control enterprise

Microsoft Foundry separa claramente **control plane** y **data plane**, y cada uno tiene su propia superficie de autenticación y RBAC. Además, en Foundry el modelo de permisos se organiza sobre dos scopes especialmente importantes:

- **Foundry resource**, como marco administrativo, de seguridad y monitorización;
- **Foundry project**, como subámbito para APIs, tools y workflows.

Agent identity: el agente como actor con identidad propia

Existen dos modos:

- **attended**, con flujo delegado **on-behalf-of**, donde el agente actúa dentro de los permisos del usuario;
- **unattended**, donde el agente actúa con su propia autoridad y RBAC

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Red privada y aislamiento: cuando importa cómo circula el dato

Microsoft Foundry Agent Service ofrece un **Standard Setup with private networking** donde llevas tu propia **private virtual network**.

Foundry Agent Service

Es una plataforma **fully managed** para construir, desplegar y escalar agentes de IA. Maneja **hosting, scaling, identity, observability and enterprise security**, para que el equipo se centre en la lógica del agente. Puede alojar **prompt agents** y **hosted agents** construidos con **Agent Framework, LangGraph o código propio**.

- **MAF** ayuda a construir la lógica agéntica,
- **Foundry Agent Service** aporta la capa gestionada de operación.

Uso profesional para proyectos que exigen más rigor que una demo o un prototipo

Primer pilar profesional: componentes con responsabilidades claras

MAF ofrece bloques fundacionales como **model clients**, **agent session**, **context providers**, **middleware**, **MCP clients**, además de **agents** y **workflows**.

Segundo pilar profesional: workflows para controlar rutas y no improvisar todo con el LLM

En cada paso puedes decidir si debe decidir el **modelo**, el **código** o un **humano**. Eso convierte al workflow en una herramienta de control, no solo de coordinación. Los workflows de MAF aportan además **type-safe routing**, **checkpointing** y soporte **human-in-the-loop**.

Tercer pilar profesional: seguridad como responsabilidad compartida

This toolbar appears only when signed in to GitBook.

Agent Framework aporta [illegible] en siendo responsables de validar inputs, asegu[illegible]mente.

Curso Azure AI
0 minutes ago

Cuarto pilar profesional: observabilidad desde el inicio

Microsoft Agent Framework integra **OpenTelemetry** y emite **traces, logs y metrics**. Además, en workflows emite spans, logs, métricas y eventos adicionales para seguir ejecución, mensajes entre executors y errores.

Quinto pilar profesional: plataforma gestionada para operar a escala

Aquí conviene separar muy bien dos capas:

- **MAF** como framework para construir lógica agéntica,
- **Foundry Agent Service** como plataforma para desplegar, alojar y escalar.

Sexto pilar profesional: gobierno, ownership y control plane

Se recomienda un enfoque muy claro:

- **single control plane**,
- **centralized agent identity**,
- **consistent policy enforcement**,
- **unified inventory and ownership**,
- y **continuous behavioral visibility**.

MAPEO CONCEPTUAL: LANGGRAPH, LANGCHAIN, MAF Y FOUNDRY

Graphs, nodes y edges frente a workflows y componentes equivalentes dentro del stack Microsoft

En LangGraph, existen:

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

- un **graph** como runti
- **nodes** como pasos o unidades de trabajo,

- **edges** como transiciones o routing,
- y **state** como contexto compartido del flujo.

En Microsoft Agent Framework, esa misma intuición aparece como:

- **workflow** para el grafo,
- **executors** y **AgentExecutor** para los nodos operativos,
- **edges** para las rutas,
- **workflow state** y **agent session** para el estado,
- y patrones de orquestación ya empaquetados para secuencia, paralelismo, handoff o group chat.

MAF describe los workflows como una **graph-based architecture** con **executors** y **edges**, routing condicional, paralelismo y paths dinámicos.

También se indica que el **Workflow Builder & Execution** une executors y edges en un **directed graph** y lo ejecuta por supersteps.

Por tanto:

- **Graph** en LangGraph → **Workflow** en MAF
- **Graph runtime** → **Workflow Builder + Execution engine**
- **Graph execution** → **supersteps + message passing + events** en MAF.

Nodes frente a Executors, Agents y AgentExecutor

En LangGraph, un **node** puede ser casi cualquier paso: una llamada al modelo, una función, una tool wrapper, un router o un paso de agregación.

En MAF, Microsoft reparte ese espacio en varias piezas:

1. Executor

Microsoft define los **executors** como los bloques fundamentales del workflow: unidades autónomas que reciben **typed messages**, ejecutan operaciones y producen mensajes o eventos. Pueden ser lógica custom o componentes basados en IA.

2. Agent

This toolbar appears only when signed in to GitBook.

Microsoft define los **agente** procesar entradas, llamar **tools** y **MCP servers**, y g

Curso Azure AI
0 minutes ago

3. AgentExecutor

Cuando quieres meter un agent dentro del workflow, Microsoft introduce **Agent Executor**. La documentación explica que esta pieza:

Edges

Aquí el mapeo es el más directo.

Microsoft documenta los **edges** en workflows como las conexiones que definen el flujo de mensajes entre executors y soportan varias formas de routing:

- **Direct**
- **Conditional**
- **Switch-case**
- **Fan-out**
- **Fan-in**.

Eso coincide muy bien con la mentalidad de LangGraph:

- edge simple,
- branching condicional,
- paralelismo,
- agregación.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Patrón	Métodos principales	Ejemplo conceptual
Secuencia simple	`add_edge` o `add_chain`	`A → B → C`
Routing condicional	`add_edge(..., condition=...)`	`Classifier → Billing / Support / General`
Switch-case	`add_switch_case_edge_group`	Primera condición que encaje gana
Fan-out	`add_fan_out_edges`	`Input → Security + Cost + Architecture`
Fan-in	`add_fan_in_edges`	`Security + Cost + Architecture → Final`
Agentes dentro del workflow	`add_agent` / `register_agent`	Un agente LLM como nodo del grafo
Producción/procesos largos	`with_checkpointing`, `set_max_iterations`	Reintentos, pausa/reanudación, control de ciclos

StateGraph frente a workflow state, agent session y context

En LangGraph, se usa `StateGraph` como referencia central: el estado es una pieza de primer nivel.

En MAF, el estado no desaparece; se reparte entre varias abstracciones:

1. Workflow state

Microsoft documenta que el **state** del workflow permite que varios executors accedan y modifiquen datos comunes cuando el paso directo de mensajes no es suficiente o no es eficiente.

2. Agent session

Para el comportamiento conversacional del agent, Microsoft usa **session-based state management** y, en Agent Executor, el executor gestiona la sesión y el estado conversacional entre runs.

3. Context providers / memoria

This toolbar appears only when signed in to GitBook.

Microsoft separa además el contexto y la memoria en proveedores específicos.

Curso Azure AI
0 minutes ago

Checkpointing: una diferencia muy importante para equipos graph-first

Los checkpoints permiten guardar el estado del workflow en puntos concretos y reanudar más tarde. También aclara que se crean al final de cada superstep y capturan:

- el estado actual de todos los executors,
- los mensajes pendientes para el siguiente superstep,
- requests y responses pendientes,
- y shared states.

Graph patterns frente a orchestrations built-in

LangGraph suele dar mucha libertad para definir topologías.
MAF, además trae patrones multiagente ya empaquetados.

Microsoft incluye como orquestaciones built-in:

- **Sequential**
- **Concurrent**
- **Handoff**
- **Group Chat**
- **Magentic**.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Orquestador	Builder típico	Patrón	Cuándo usarlo
Sequential	`SequentialBuilder`	Pipeline lineal: agente 1 → agente 2 → agente 3	Cuando cada paso depende del anterior: redacción → revisión → síntesis, clasificación → análisis → respuesta final
Concurrent	`ConcurrentBuilder`	Paralelismo: varios agentes trabajan sobre la misma entrada	Cuando quieres varias perspectivas independientes: seguridad, coste, arquitectura, legal, calidad
Handoff	`HandoffBuilder`	Delegación dinámica entre agentes	Soporte, sistemas expertos o flujos donde el agente actual decide transferir a otro especialista
Group Chat	`GroupChatBuilder`	Conversación colaborativa coordinada por un manager	Refinamiento iterativo, debate entre agentes, análisis multi-perspectiva con turnos controlados
Magentic	`MagenticBuilder`	Manager LLM planifica, delega, monitoriza y replanifica	Tareas abiertas y complejas donde no conoces de antemano la ruta: research, análisis, coding, informes largos

Dónde entra Foundry en este mapa

`AgentServiceFactory` crea **LangGraph-compatible nodes** que corren a través de Agent Service y pueden componer soluciones más complejas con LangGraph.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: WorkFlowBuilder

az207.py

```
import asyncio
from typing_extensions import Never

from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class ClassifierExecutor(Executor):
    @handler
    async def classify(self, text: str, ctx: WorkflowContext[str]) -> None:
        lower = text.lower()
```

Ejercicio: Experimentar con Patrones en el Chat de Soporte

Estado y checkpointing comparados con las opciones de persistencia y control disponibles

Conviene fijar cuatro conceptos:

Estado es el contexto operativo que el sistema necesita ahora para seguir funcionando.
Persistencia es el mecanismo por el que ese contexto o esos datos sobreviven al turno actual o incluso al reinicio del proceso.
Checkpoint es una instantánea del progreso de un workflow para poder **reanudarlo** o **recuperarlo** sin perder lo ya hecho.
Memoria es conocimiento útil que el agente conserva o recupera entre sesiones para decidir mejor en el futuro.

Opción 1: estado local de sesión en MAF

Microsoft Agent Framework incluye dos modos regulares de almacenamiento conversacional. El primero es **Local session state**, donde el historial completo vive en `AgentSession.state`, por ejemplo mediante `InMemoryHistoryProvider`

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Microsoft lo presenta como adecuado para servicios que **no requieren persistencia conversacional del lado del servidor**. Además, cuando el proveedor no necesita historial server-side, Agent Framework mantiene el historial localmente en la sesión y reenvía los mensajes relevantes en cada ejecución.

Opción 2: service-managed storage para conversación

El segundo modo que incluye MAF es **Service-managed storage**. Ahí el estado de la conversación vive en el servicio y `AgentSession.service_session_id` apunta a esa persistencia.

Microsoft lo recomienda para servicios con **native persistent conversation support**.

Ésta ya no es una simple variable local:

- el estado conversacional pasa a estar gestionado por la capa de servicio,
- la sesión tiene una representación durable fuera del proceso,
- y la reanudación ya no depende solo de mantener un objeto en memoria local.

Opción 3: checkpointing en workflows de MAF

Los **checkpoints** de workflows son snapshots del estado del workflow que permiten guardarlo en puntos concretos y reanudarlo más tarde.

Se crean al final de cada **superstep** y que capturan:

- el estado actual de todos los executors,
- los mensajes pendientes para el siguiente superstep,
- las requests y responses pendientes,
- y los **shared states**.

Opción 4: implementaciones de almacenamiento de checkpoints

Hay dos implementaciones claras para Python:

- **InMemoryCheckpointStorage**, que Microsoft describe como almacenamiento de checkpoints en memo
- **FileCheckpointStora** nto de checkpoints **basado en ficheros** p

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Y además `WorkflowBuilder.with_checkpointing(...)` acepta una abstracción `CheckpointStorage`, de forma que la estrategia concreta de persistencia queda desacoplada del workflow.

Opción 5: Durable execution con Durable Task

Microsoft incluye una **Durable Task extension for Microsoft Agent Framework (Preview)** que lleva la ejecución durable directamente a MAF.

Puedes registrar agentes para hacerlos automáticamente durables con:

- **persistent sessions**,
- **built-in API endpoints**,
- y **distributed scaling**,

Internamente implementa **entity-based agent loops**, donde cada sesión del agente es una entidad durable que gestiona automáticamente estado conversacional y checkpointing.

Opción 6: persistencia conversacional en Foundry Agent Service

En Foundry aparece otra capa de persistencia. La documentación del nuevo servicio de agentes indica como beneficio enterprise:

- **single-tenant storage**,
- opción de **bring your own Azure Cosmos DB** para almacenar estado,
- y **stateful context** que retiene automáticamente contexto entre llamadas.
  Además, la migración al nuevo modelo reemplaza **threads** por **conversations** y **runs** por **responses**.

Los **threads** persistían hasta borrado explícito y, en el **Standard agent setup**, se almacenaban en tu cuenta de **Azure Cosmos DB**, con implicaciones directas sobre coste y cumplimiento.

Opción 7: memoria de largo plazo en Foundry

La memoria es **persistent knowledge retained by an agent across sessions** y que, en general, hay dos categorías:

This toolbar appears only when signed in to GitBook.

- **short-term memory**, el contexto de sesión,
- y **long-term memory** e, consolida y gestiona conocimiento a lo largo del tiempo.

Curso Azure AI
0 minutes ago

Foundry Memory está diseñada para long-term memory y no para sustituir la gestión de sesión del framework.

- usar memory para **user-specific context that persists over time**,
- usar una **knowledge base** para grounding organizacional,
- y usar **file search** para documentos aportados en una interacción.

Comparación con LangGraph

Al usar **LangGraph con Foundry Agent Service**, el parámetro `agent` puede ser un `CompiledStateGraph`, y para usar **persistent checkpointing** debes compilar el grafo con un **checkpointer** mediante `builder.compile(checkpointer=saver)`.

Conceptos clave

1. Estado de sesión

Es el contexto operativo inmediato de una conversación o ejecución. En MAF puede vivir localmente en `AgentSession.state` o estar gestionado por el servicio.

2. Persistencia conversacional

Es la capacidad de retener el historial de interacción entre llamadas. Puede ser local o service-managed.

3. Checkpoint

Es una instantánea del progreso del workflow para pausar, reanudar o recuperarse de fallos. Captura estado de executors, mensajes pendientes, requests/responses y shared state.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: workflow con checkpointing en fichero

```
from agent_framework import WorkflowBuilder, FileCheckpointStorage

storage = FileCheckpointStorage("./checkpoints")

workflow = (
    WorkflowBuilder(checkpoint_storage=storage)
    # .register_executor(...)
    # .add_edge(...)
    # .set_start_executor(...)
    .build()
)
```

Listar Checkpoints

```
checkpoints = await checkpoint_storage.list_checkpoints(
    workflow_name=workflow.name
)

for cp in checkpoints:
    print(cp.checkpoint_id)
```

Retomar Checkpoints

```
checkpoints = await checkpoint_storage.list_checkpoints(
    workflow_name=workflow.name
)

if not checkpoints:
    raise RuntimeError("No hay checkpoints disponibles.")

latest = max(checkpoints, key=lambda cp: cp.timestamp)

async for event in workflow.run(
    checkpoint_id=latest.checkpoint_id,
```

Ejercicio. Probar Checkpointing

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Tools y function calling como mecanismo común con diferencias de implementación y gobierno

Azure OpenAI lo resuelve en tres pasos: enviar tools con la entrada del usuario, ejecutar en tu aplicación la función que el modelo solicite y volver a llamar al modelo con la salida de la tool. Además, desde la API `2023-12-01-preview`, `functions` y `function_call` quedaron sustituidos por `tools` y `tool_choice`

En Foundry Agent Service, se distinguen dos grandes familias: **built-in tools** y **custom tools**.

Azure OpenAI: el nivel más básico del patrón

En Azure OpenAI, function calling es un mecanismo de inferencia relativamente “bajo nivel”.

Es un flujo donde tu app declara tools, el modelo puede devolver una solicitud de llamada, tu backend ejecuta la función y luego llamas otra vez al modelo con el resultado.

Además, la **Responses API** de Azure OpenAI unifica capacidades conversacionales y stateful multi-turn responses, y por defecto retiene datos de respuesta durante 30 días salvo borrado explícito.

MAF: function calling como abstracción de framework

En Microsoft Agent Framework, las function tools dejan de ser solo un detalle del payload al modelo y pasan a ser una abstracción de primer nivel del framework.

Puedes convertir funciones en tools y `AIFunction` envuelve una función para hacerla callable por modelos con **automatic parameter validation** y **JSON schema generation**.

Los parámetros tipados se convierten en JSON Schema que el agente usa para pasar argumentos.

Además:

- las function tools son **custom code** que el agente puede llamar cuando hace falta,
- y **no todos los tipos d**[illegible] o, `ChatClientAgent` sí las soporta, pero no [illegible] ilidad

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Aprobación humana: una diferencia importante de gobierno

MAF incluye **approval_mode**. Puedes marcar una función con `approval_mode="always_require"` para que el agente no complete la acción directamente, sino que emita una solicitud de aprobación y la aplicación recoja esa respuesta del usuario antes de continuar.

MCP: tools como protocolo, no solo como funciones locales

Microsoft Agent Framework y Foundry también entran en el mundo **MCP (Model Context Protocol)**. MCP es un estándar abierto para que las aplicaciones proporcionen tools y datos contextuales a los LLMs.

MAF incluye herramientas MCP hosteadas por Foundry y clases específicas como `HostedMCPTool`, `MCPStdioTool`, `MCPWebsocketTool` o `MCPStreamableHTTPTool`, varias de ellas con soporte para `approval_mode` y `allowed_tools`.

Foundry Agent Service: tools como producto de plataforma

Microsoft Foundry Agent Service ofrece:

- **built-in tools** que la plataforma ejecuta y gestiona,
- **custom tools** para llevar capacidades propias,
- y un **tool catalog** general availability para descubrir y configurar herramientas, aunque algunas herramientas concretas siguen en preview.

Algunas built-in habituales:

- web search,
- code interpreter,
- file search,
- function calling,

y como custom tools frecuentes:

- MCP,
- A2A,
- OpenAPI.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Diferencias prácticas de implementación

Azure OpenAI

- Tú declaras `tools`.
- El modelo decide si llamarlas.
- Tu app ejecuta la función.
- Tu app devuelve el resultado al modelo.
- El gobierno depende casi por completo de tu backend.

MAF

- Sigues teniendo function calling, pero envuelto en abstracciones como `AIFunction`.
- Los tipos se convierten en JSON Schema.
- Puedes usar `approval_mode`.
- Puedes integrar tools locales o MCP.
- El framework ayuda a ordenar contratos, validación y seguridad.

Foundry Agent Service

- Añade catálogo de tools, built-ins y custom tools.
- La plataforma puede gestionar ejecución de herramientas built-in.
- MCP y otras integraciones tienen controles de allow-list y approval.
- Algunas decisiones pasan del código a la configuración y al gobierno del servicio.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: `tools` en OpenAI vs. @tool vs AIFunction

```
from openai import OpenAI
import os
import json

client = OpenAI(
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    base_url=f"{os.getenv('AZURE_OPENAI_ENDPOINT')}/openai/v1/",
)

tools = [
    {
```

```
from typing import Annotated
from pydantic import Field
from agent_framework import tool

@tool(approval_mode="always_require")
def create_ticket(
    title: Annotated[str, Field(description="Título corto de la incidencia")],
    priority: Annotated[str, Field(description="Prioridad: baja, media o alta")]
) -> str:
    allowed = {"baja", "media", "alta"}
    if priority not in allowed:
```

```
from typing import Annotated
from pydantic import BaseModel, Field
from agent_framework import AIFunction


class WeatherArgs(BaseModel):
    location: Annotated[
        str,
        Field(description="Ciudad que se quiere consultar")
    ]
```

Ejercicio: Migrar @tool a AIFunction

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Handoffs y delegación entre agentes o componentes según patrón elegido

Microsoft Agent Framework diferencia con bastante claridad:

- **agents as tools**, donde un agente externo sigue mandando y usa a otro como una herramienta,
- **handoff**, donde el control y la propiedad del caso se transfieren a otro agente,
- y **workflows**, donde el grafo define la coordinación de forma explícita.

Primer patrón: agent-as-tool, delegación ligera y model-driven

Agents as tools es el patrón en el que un agente externo puede llamar a otro agente igual que llamaría a una function tool.

Existen varios rasgos importantes:

- la delegación es **model-driven**, no hard-coded;
- el agente externo decide **cuándo** llamar al agente interno;
- el agente externo solo ve la **respuesta final** del agente interno;
- el agente interno no hereda automáticamente toda la conversación exterior, sino solo lo que se le pase como argumentos.

Segundo patrón: handoff, cesión real del control

En handoff orchestration:

- el control se pasa entre agentes según reglas definidas;
- cada agente puede decidir transferir la tarea completa a otro;
- y no existe una autoridad central que siga dirigiendo el caso después de cada subtarea.

Existen tres diferencias clave frente a agents-as-tools:

Control flow
En handoff, el control se transfiere entre agentes. En agent-as-tools, el control vuelve al agente principal tras cada subtarea.

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Task ownership
En handoff, el agente receptor toma la **propiedad completa** del caso. En agent-as-tools, el agente principal conserva la responsabilidad global.

Context management
En handoff, la conversación se entrega al otro agente con el contexto del caso. En agent-as-tools, el agente principal filtra y pasa solo lo que considere necesario.

Tercer patrón: agents in workflows, delegación definida por el grafo

En workflows, el grafo define la ruta. Si la pregunta es “qué subtarea va después” y la respuesta es “el modelo”, usa agents-as-tools; si la respuesta es “el desarrollador”, usa workflows.

Ejemplo: handoff como cesión de ownership

az208.py

```
import asyncio
import os
from pathlib import Path

from dotenv import load_dotenv
from agent_framework import Agent

try:
    from agent_framework.orchestrations import HandoffBuilder
except ImportError:
    from agent_framework import HandoffBuilder
```

Ejercicio. Convertir en Agent-As-Tool

```
coordinator = Agent(
    client=client,
    name="coordinator",
    instructions=(
        "Eres un coordinador de soporte de nivel 1. "
        "Decide si debes responder directamente o delegar en una tool
especializada. "
        "Cuando recibas el resultado de una tool, sintetiza una respuesta final
para el usuario."
    ),
    tools=[
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Memoria de corto y largo plazo y cómo repartir responsabilidades entre framework y plataforma

La **short-term memory** sigue normalmente en el contexto de sesión del framework, mientras que **Foundry Memory** está diseñada para **long-term memory**, es decir, conocimiento persistente que el agente puede recuperar en sesiones futuras.

Conviene separar cuatro conceptos:

Estado
Es el contexto operativo que el sistema necesita ahora mismo para continuar.

Persistencia conversacional
Es el mecanismo por el que la conversación puede sobrevivir a varias llamadas o turnos.

Checkpointing
Es una instantánea del progreso del workflow para poder reanudarlo tras pausa, reinicio o fallo.

Memoria de largo plazo
Es conocimiento útil que se conserva entre sesiones y que no depende solo del turno actual.

Persistencia conversacional: a veces local, a veces gestionada por servicio

Local session state
Encaja cuando no necesitas persistencia conversacional server-side y quieres simplicidad o control local. El historial completo vive en la sesión.

Service-managed storage
Encaja cuando el servicio tiene soporte nativo de conversación persistente. En ese caso, la sesión apunta a una conversación gestionada por el servicio.

Checkpointing

En Workflows, Microsoft define los checkpoints como snapshots del estado del workflow que permiten **pause/resume** y **recovery from failures**.

En la clase `Workflow` , cu

This toolbar appears only when signed in to GitBook.

points se crean al final de cada **superstep** y captura

Curso Azure AI
0 minutes ago

- **executor states**,

- **messages in transit**,
- y **shared state**. También permite pausar y reanudar workflows tras reinicios de proceso.

Memoria de largo plazo

Memory es una solución **managed, long-term memory**. Permite continuidad del agente **across sessions, devices, and workflows**

Foundry Memory:

- extrae información significativa de las conversaciones,
- la consolida en conocimiento durable,
- y la hace disponible en sesiones futuras.

Foundry Memory introduce el concepto de **scope** para segmentar la memoria entre usuarios y contextos de forma segura y aislada:

- **user ID** como identidad estable para memoria de largo plazo,
- **session ID** como identidad de corto plazo para una conversación concreta,
- y otros scopes como resource IDs cuando hace falta memoria más compartida o transversal.

Foundry Agent Service añade otra capa: contexto persistente del servicio

La nueva Foundry Agent Service añade beneficios que afectan también a este reparto:

- **single-tenant storage** con opción de usar tu propio Azure Cosmos DB para guardar estado,
- **stateful context** que retiene automáticamente contexto entre llamadas,
- y una gestión más moderna basada en **conversations** y **responses**.

Durable execution

La **Durable Task extension for Microsoft Agent Framework**. El foco ya no es solo recordar contexto, sino hacer durable la ejecución del agente. La documentación indica que las sesiones durables mantienen automáticamente historial y estado, y añade una **TTL** para limpiar sesiones inactivas y evitar consumo indefinido de almacenamiento.

Cuando una sesión exced [illegible] na y, si vuelve a llegar un mensaje, se crea una se [illegible]

This toolbar appears only when signed in to GitBook.
Curso Azure AI
0 minutes ago

Esto introduce otra distinción muy útil:

- **Memory de largo plazo** sirve para recordar conocimiento útil.
- **Durable execution** sirve para que la sesión o la ejecución sobrevivan con garantías operativas.

Human-in-the-loop para aprobaciones, supervisión o intervención en puntos críticos

En workflows, los ejecutores pueden enviar solicitudes fuera del flujo y esperar respuestas mediante `RequestPort` o `ctx.request_info()`, y en tools puedes marcar funciones para que requieran aprobación humana antes de ejecutarse.

Además, las orquestaciones soportan HITL mediante **tool approval** y **request info**.

HITL es el patrón adecuado cuando el sistema debe **esperar input externo** antes de seguir, y Foundry lo incluye como patrón de workflow para **approval requests** o para obtener información del usuario en medio de la ejecución.

En práctica conviene separar tres variantes:

Aprobación
El agente sabe qué quiere hacer, pero necesita permiso humano antes de ejecutar la acción. Éste es el caso típico de **tool approval**

Supervisión
La persona no necesariamente bloquea cada paso, pero revisa decisiones, rutas o resultados en momentos sensibles. Esto suele combinarse con observabilidad, trazas y eventos del workflow.

Intervención
El workflow no solo pide permiso, sino información nueva sin la cual no puede seguir. En MAF esto se modela con **request/response handling** usando `RequestPort` o `ctx.request_info()`.

Primer patrón: aprobación humana de tools

En este enfoque, si una tool necesita revisión humana, puedes marcarla para que requiera aprobación antes de ejecu

This toolbar appears only when signed in to GitBook.

En Python se hace con

Curso Azure AI
0 minutes ago

Cuando el agente necesita esa aprobación, la ejecución no termina con una respuesta final, sino con una respuesta que indica **qué input humano falta**. Después, el caller debe recoger la decisión del usuario y reenviarla al agente en un nuevo run.

Segundo patrón: request/response en workflows

Los ejecutores pueden enviar solicitudes a sistemas externos y esperar respuesta antes de continuar.

En Python, se usa `ctx.request_info()` y un método marcado con `@response_handler` para procesar la respuesta.

Cuando el workflow emite una solicitud, genera un `RequestInfoEvent` que sistemas externos o una UI pueden escuchar; cuando llega la respuesta, el framework la enruta automáticamente al ejecutor correcto.

HITL dentro de orquestaciones y workflows más amplios

HITL no es un patrón aislado, sino una capacidad transversal que puede aparecer dentro de secuencias, handoffs, procesos multiagente o workflows híbridos.

- al principio, para completar información,
- en medio, para revisar un paso crítico,
- o cerca del final, para autorizar una acción irreversible.

Ejemplo: procesar una solicitud de aprobación

```
result = await agent.run("Aprueba el reembolso REQ-123 por 350 euros")

if result.user_input_requests:
    for user_input_needed in result.user_input_requests:
        print("Tool:", user_input_needed.function_call.name)
        print("Args:", user_input_needed.function_call.arguments)

        # Simulación de aprobación humana
        approval_message = input("Aprobado?")

        final_result = await agent.run([
```

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Ejemplo: workflow con `ctx.request_info()`

```
from dataclasses import dataclass
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler,
response_handler

@dataclass
class ApprovalRequest:
    document_id: str
    reason: str

class HumanReviewExecutor(Executor):
    def __init__(self):
```

Tracing y debugging para diagnosticar decisiones, errores y rutas de ejecución

Tracing
Es el registro distribuido de la ejecución: spans, atributos, duración, tokens, tool calls y relaciones entre operaciones. MAF lo integra con OpenTelemetry.

Workflow events
Son señales del runtime del workflow emitidas en puntos clave y consumibles por streaming. Sirven para seguir el progreso del flujo, detectar errores parciales y exponer eventos custom.

Debugging
Es la actividad de análisis: usar trazas, eventos, métricas y vistas de Application Insights o Foundry para aislar la causa de un problema. Foundry describe el tracing precisamente como herramienta para debug y monitorización en producción.

Microsoft Agent Framework incluye observabilidad integrada con **OpenTelemetry** y muestra varias formas de activarla, incluyendo `configure_otel_providers()` , exportadores OTLP personalizados e integración con Azure Monitor.

Además, MAF expone wrappers para crear spans y métricas custom con `get_tracer()` y `get_meter()`

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Qué traza automáticamente MAF

- `invoke_agent <agent_name>` como span raíz de una invocación, con atributos como instrucciones y uso de tokens.
- `execute_tool <function_name>` cuando el agente llama a una function tool.
- métricas como `gen_ai.client.operation.duration` y `gen_ai.client.token.usage` para duración y uso de tokens.

Diferencias operativas al pasar de un stack open source a uno más gestionado

Hosting y escalado

Microsoft Foundry Agent Service es una plataforma **fully managed** que se ocupa de hosting y scaling.

Identidad y acceso

Microsoft recomienda usar **Microsoft Entra ID** para producción porque habilita acceso condicional, identidades administradas y RBAC de privilegio mínimo, mientras que las **API keys** quedan como opción válida para prototipos rápidos, pero poco granular y menos auditable

Observabilidad

El tracing en Foundry almacena la telemetría en **Azure Monitor Application Insights** y captura entradas, salidas, tool usage, latencias, costes y consumo de tokens.

Además, Application Insights ofrece una vista **Agent details** para monitorizar agentes de Foundry, Copilot Studio y terceros, y permite seguir rendimiento, errores y costes desde una superficie más operativa.

Red y aislamiento

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

En Foundry Agent Servic **vate networking** donde tú aportas la VNet privada, pero la plataforma crea un entorno aislado y, por defecto, garantiza:

Estado, memoria y persistencia

En Foundry Agent Service, aparece una capa gestionada. **Memory** como una solución **managed, long-term memory** que conserva conocimiento útil entre sesiones

Gobierno e inventario

Microsoft Foundry Control Plane documenta **centralized management and observability** para agentes distribuidos en varios proyectos de una suscripción.

Trade-offs técnicos entre flexibilidad máxima y capacidades enterprise integradas

La principal ventaja de un stack muy flexible es que puedes decidir con mucha precisión:

- qué framework usas,
- cómo modelas el workflow,
- cómo serializas estado,
- qué backend de memoria eliges,
- cómo instrumentas telemetría,
- y hasta en qué entorno y con qué topología ejecutas cada componente.

Integración híbrida

Foundry Agent Service permite usar **prompt agents** desde el portal, pero también **hosted agents** construidos con **Agent Framework, LangGraph o código propio**. Además, Foundry Control Plane puede registrar agentes custom de otras plataformas e infraestructuras para darles visibilidad y gestión centralizada.

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Decisiones de arquitectura para elegir qué construir y qué delegar a la plataforma

Se recomienda escoger entre **code-first frameworks** y **managed orchestration** según la capacidad de ingeniería, las necesidades de personalización y la carga operativa que la organización está dispuesta a asumir.

Qué suele merecer la pena desarrollar

Hay varias piezas que suelen justificar un enfoque más framework-first o más custom, porque son las que más valor diferencial aportan.

Lógica de negocio y criterios de decisión

La lógica que traduce reglas reales del negocio, priorización, composición de pasos, selección de herramientas, fallback y criterios de salida suele ser donde más valor aporta el equipo.

Ésa es una buena candidata para quedarse en MAF, LangGraph o código propio, porque es la parte más específica del caso. Microsoft Foundry Agent Service está pensado para que te centres precisamente en esa lógica mientras la plataforma absorbe capas más operativas.

Orquestación singular o muy específica

Si el flujo tiene una topología muy particular, mezcla varios patrones de coordinación o requiere un control muy fino del runtime, un enfoque code-first suele encajar mejor.

Integraciones muy propias

APIs corporativas, reglas transaccionales, conectores internos, normalización de datos y adaptadores a sistemas legados suelen ser piezas que conviene construir y gobernar en tu capa de aplicación.

Qué suele merecer la pena delegar a la plataforma

Hosting y escalado

Identidad, autenticación y

This toolbar appears only when signed in to GitBook.

Observabilidad y super

Curso Azure AI
0 minutes ago

Cómo decidir con un criterio simple

Una forma práctica de tomar la decisión es pasar cada pieza por cuatro preguntas.

1. ¿Me diferencia funcionalmente?

2. ¿Es transversal a muchos agentes?

3. ¿Es costosa de operar y no aporta diferenciación clara?

4. ¿Necesito control extremo o portabilidad real?

DISCUSION PROYECTOS FINALES

Soporte L1 con documentación de proyecto + escalado a JIRA

Un agente de soporte de nivel 1 que identifique proyecto, consulte documentación en SharePoint y, si no encuentra solución, registre la incidencia en JIRA con el detalle recopilado.

Agente asíncrono en background basado en eventos

El agente elige la acción y el **workflow** gestiona integración, retries, pasos largos y conectores.

Deep agent con múltiples ficheros, razonamiento y salida no textual

Un deep agent que reciba varios ficheros, combine conocimiento interno o skills y produzca como salida un fichero o a

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago

Copiloto interno gobernado sobre conocimiento corporativo

RAG sobre conocimiento corporativo, integración con sistemas internos y construcción de agentes seguros y gobernados.

Cuestionario Sesión 2

Haz click aquí ↗ cuando lo indique el formador

Feedback Sesión 2

Haz click aquí ↗ cuando lo indique el formador

<

Previous
Sesión 1

Last updated 6 hours ago

This toolbar appears only when signed in to GitBook.

Curso Azure AI
0 minutes ago